



Computer Science Department
Summer Internship Report
Second Year in Software Engineering
Presented by: **Yassine Ben Jemaa**

**Design and Implementation of a
Real-Time Anomaly Detection Platform
for Branch Teller Operations
using Deep Learning**

Supervisor: **Mr. Oussama Ben Cheikh**

Realized within:



Academic Year: 2025–2026

Acknowledgements

I would like to express my sincere gratitude to Amen Bank for offering me the opportunity to complete this internship within the Direction Centrale du Système d'Information. I would especially like to thank my supervisor, Mr. Oussama Ben Cheikh, for his guidance and for granting me the freedom to explore beyond the original project scope.

This experience allowed me to acquire practical skills in deep learning, streaming architectures, and MLOps, and to discover the challenges of building production-oriented systems in a real banking environment.

I would also like to thank the Faculté des Sciences de Tunis and the University of Tunis El Manar for the academic foundation that made this work possible.

Contents

Acknowledgements	i
List of Figures	iii
List of Tables	iv
List of Abbreviations	v
General Introduction	1
1 Context and State of the Art	3
1.1 Host Institution	3
1.2 Branch Teller Operations	3
1.3 Regulatory Framework	4
1.3.1 Banque Centrale de Tunisie (BCT)	4
1.3.2 Commission Tunisienne des Analyses Financières (CTAF)	4
1.3.3 Law n°41-2024	5
1.4 Existing Detection Capabilities	5
1.5 State of the Art	5
1.5.1 Rule-Based Detection	5
1.5.2 Statistical Anomaly Detection	6
1.5.3 Autoencoders for Anomaly Detection	6
1.5.4 LSTM Networks for Sequential Anomaly Detection	7
1.5.5 Score Fusion	7
1.5.6 Explainability with SHAP	8
1.5.7 Streaming Architectures for Real-Time Detection	8
1.6 Problem Statement and Project Objectives	9
2 System Architecture and Design	10
2.1 System Overview	10
2.2 Streaming Topology	11
2.3 Hot/Cold Data Layer	12
2.4 Raw Event Schema	12

2.5	Dual-Actor Profiling	13
2.5.1	Client Profile	13
2.5.2	Employee Profile	14
2.6	Enriched Event Design	15
2.6.1	The Profile Dominance Problem	15
2.6.2	Contrast Feature Redesign	15
2.7	Scoring Architecture	16
2.7.1	Dual-Model Rationale	16
2.7.2	Score Fusion	17
2.7.3	Cold-Start Scoring	17
2.7.4	Conditional SHAP	18
2.8	Decision Service	18
2.8.1	Layer 1 — Score Thresholds	18
2.8.2	Layer 2 — Regulatory Rule Escalation	19
2.8.3	Layer 3 — Risk History Adjustment	19
2.8.4	Explanation Assembly	20
2.9	Software Architecture	20
3	Synthetic Data Engineering	22
3.1	Source Extraction Methodology	22
3.2	Source Extractions	23
3.2.1	S1 — PaySim	23
3.2.2	S2 — Amen Bank 2023 Annual Report	23
3.2.3	S3 — CTAF 2021 Annual Report	24
3.2.4	Known Gap — BCT Circulars	24
3.3	Client Archetype Design	25
3.3.1	Regulatory Discovery — Law n°41-2024	25
3.4	Generator Architecture	25
3.4.1	Branch-Linked Employee Assignment	26
3.5	Anomaly Scenario Design	27
3.6	Data Validation	27
3.6.1	Oracle Profile Shortcut	28
4	Model Training and Evaluation	30
4.1	Data Preparation	30
4.1.1	Temporal Split	30
4.1.2	Class Imbalance	31
4.2	Autoencoder	31
4.2.1	Architecture	31
4.2.2	Anomaly Scoring	31

4.3	LSTM	32
4.3.1	Architecture	32
4.3.2	Anomaly Scoring	32
4.3.3	Sequence Construction	32
4.4	Initial Training Failure and Root Cause Analysis	32
4.4.1	Diagnosis	33
4.4.2	Resolution	33
4.5	Post-Rebuild Results	34
4.5.1	Per-Scenario Analysis	34
4.5.2	Complementary Detection Profiles	35
4.6	Framing for Production Deployment	35
4.6.1	Multi-Mechanism Convergence: Smurfing as Case Study	35
5	Streaming Pipeline and Integration	37
5.1	Streaming Topology	37
5.1.1	Broker Choice: RedPanda	38
5.1.2	Consumer Groups and Offset Management	38
5.2	Core Extraction Pattern	38
5.2.1	Core Modules	39
5.2.2	Thin Streaming Wrappers	39
5.3	Dead Letter Queue and Fault Tolerance	39
5.3.1	Failure Classification	39
5.3.2	ResilientConsumer	40
5.3.3	Structured Logging	41
5.4	Enrichment Service	41
5.4.1	Data Sources	41
5.4.2	Feature Computation	42
5.4.3	Amount Statistics Fallback Chain	42
5.4.4	Sequence Buffer Management	42
5.5	Scoring Service	43
5.5.1	Score Fusion	43
5.5.2	Cold-Start Guard	43
5.5.3	Conditional SHAP	44
5.6	Decision Service	44
5.6.1	Alert Tier Assignment	44
5.6.2	Regulatory Rules	44
5.6.3	Cold-Start Tier Cap	45
5.7	Sink Consumers	45
5.7.1	Raw Events Sink	45

5.7.2	Decisions Sink	46
5.7.3	Deliberate FK Omission	46
5.8	Data Persistence Layer	46
5.8.1	Master Tables	48
5.8.2	Transaction and Event Tables	49
5.8.3	Alerts Table	50
5.8.4	Profile and Baseline Tables	50
5.9	Batch Integration	51
5.9.1	Nightly Profile Rebuild	52
5.9.2	Feature Parity Guarantee	52
5.9.3	Streaming-Batch Coordination	52
6	MLOps, Observability, and Production Readiness	54
6.1	MLOps Lifecycle Overview	54
6.2	Experiment Tracking with MLflow	55
6.2.1	Server Configuration	55
6.2.2	Training Job Structure	55
6.3	Model Promotion Gate	55
6.3.1	Promotion Logic	56
6.3.2	Scope and Limitations	56
6.4	Orchestration with Apache Airflow	56
6.4.1	Airflow Infrastructure	57
6.4.2	Weekly Retrain DAG	57
6.4.3	Training Isolation with DockerOperator	57
6.5	Retraining Triggers	58
6.5.1	Scheduled Retraining	58
6.5.2	Drift-Triggered Retraining	58
6.5.3	Feedback-Triggered Retraining	59
6.6	Supervisor Feedback Loop	60
6.6.1	Feedback Divergence DAG	60
6.6.2	Three Triggers, One Pipeline	60
6.7	Observability and Monitoring	61
6.7.1	Service Metrics	61
6.7.2	Failure Domain Separation	61
6.7.3	Structured Logging	61
7	Dashboard and Results	63
7.1	Dashboard Architecture	63
7.1.1	Why the Dashboard Cannot Consume Kafka Directly	63
7.1.2	Data Access Pattern	63

7.1.3	Real-Time Refresh	64
7.1.4	Navigation Structure	64
7.2	Alerts Page	64
7.2.1	Filtering and List View	64
7.2.2	Alert Detail	65
7.2.3	Supervisor Feedback	65
7.3	KPIs Page	65
7.4	Simulation Page	66
7.5	Health Page	66
7.6	Pipeline Validation and Results	67
7.6.1	Score Distribution	67
7.6.2	Model Performance	67
7.6.3	Synthetic Data Caveat	68
8	Production Roadmap	69
8.1	Container Orchestration with Kubernetes	69
8.2	Continuous Integration and Deployment	69
8.3	Secrets Management	70
8.4	Load Testing	70
8.5	Data Retention and Archival	71
	General Conclusion	72
	A Enriched Feature Schema	75
	B Docker Compose Service Inventory	77

List of Figures

5.1	Streaming topic chain	37
5.2	Database schema — entity relationship diagram. Solid arrows represent foreign key constraints; the dashed arrow between alerts and transactions indicates a logical link without a database-level constraint (Section 5.7). . .	47
6.1	Drift-triggered retraining chain. The webhook relay exists because Alert-Manager cannot natively authenticate against Airflow’s REST API. . . .	59

List of Tables

2.1	Common envelope fields	13
2.2	Enriched event feature schema (30 features)	16
2.3	Cold-start scoring progression	17
2.4	Alert tier thresholds	18
2.5	Regulatory escalation rules	19
2.6	Synchronous endpoint design evaluation	20
3.1	Client archetypes	25
3.2	Anomaly scenarios by difficulty tier	27
3.3	Generator output files	28
3.4	Dataset validation	28
4.1	Data split strategy	30
4.2	Model performance comparison	34
4.3	Per-scenario detection performance (rebuilt models)	34
5.1	Core modules and their responsibilities	39
5.2	DLQ envelope fields	40
5.3	Enrichment Service data sources	41
5.4	Enriched feature categories	42
5.5	Scoring behavior by buffer maturity	44
5.6	Score-to-tier mapping	44
5.7	<code>clients_master</code> — one row per client	48
5.8	<code>employees_master</code> — one row per teller	48
5.9	<code>transactions</code> — populated by the raw events sink	49
5.10	<code>evaluation_labels</code> — ground truth for model evaluation	49
5.11	<code>alerts</code> — populated by the decisions sink	50
5.12	Profile and baseline tables	51
6.1	Retraining trigger comparison	60
7.1	Alert tier distribution on the synthetic dataset	67
7.2	Model AUC-ROC on the 30-feature schema	67
A.1	Complete enriched feature schema (30 features)	75

B.1 Docker Compose service inventory	77
--	----

List of Abbreviations

AE	Autoencoder
AML	Anti-Money Laundering
AUC	Area Under the Curve
BCT	Banque Centrale de Tunisie
CTAF	Commission Tunisienne des Analyses Financières
DI	Dependency Injection
DLQ	Dead Letter Queue
DT	Dinar Tunisien
E2E	End-to-End
FK	Foreign Key
FT	Financing of Terrorism
KPI	Key Performance Indicator
LAB	Lutte Anti-Blanchiment
LSTM	Long Short-Term Memory
MLOps	Machine Learning Operations
NLG	Natural Language Generation
ROC	Receiver Operating Characteristic
SHAP	SHapley Additive exPlanations

General Introduction

The digital transformation of retail banking has shifted the volume and velocity of financial transactions well beyond what manual oversight can monitor. In a network of 150 branches processing thousands of teller operations daily, the gap between transaction speed and supervisory capacity creates exposure to fraud, money laundering, operational errors, and behavioral anomalies that may go undetected until periodic compliance reviews surface them — often weeks or months after the fact.

Amen Bank’s Direction Centrale du Système d’Information already operates an AI-based anti-money laundering system oriented toward bank-wide compliance reporting. This internship addresses a complementary need: real-time decision support for branch supervisors at the point of transaction. The objective is to design and build a platform that ingests teller operations as they occur, profiles both the client and the employee involved, detects anomalies using deep learning models, and surfaces explainable alerts that a non-technical supervisor can understand and act upon immediately.

The platform covers four teller-window operations — retrait, versement, virement, and remise de chèques — and monitors two independent actors per transaction: the client initiating it and the employee processing it. An Autoencoder detects point anomalies in individual transactions while an LSTM network identifies sequential deviations in client behavior over time. Every alert is accompanied by a human-readable explanation generated through SHAP, bridging the gap between model output and supervisory action.

Beyond the detection models themselves, the project invests in the infrastructure that makes a machine learning system operational and maintainable: a streaming pipeline built on RedPanda, behavioral profiles maintained through a hot/cold data layer, automated experiment tracking with MLflow, three independent retraining triggers, structured logging, and Prometheus-based observability. The system runs as 20 Docker containers orchestrated through Docker Compose, validated end-to-end on a synthetic dataset of 243,000 transactions generated from five client archetypes calibrated against the bank’s published data.

This rapport is organized as follows. Chapter 1 presents the institutional context, the regulatory framework governing anti-money laundering in Tunisia, and a review of the anomaly detection techniques that inform the system design. Chapter 2 describes the overall architecture, the data schemas, and the key design decisions. Chapter 3 covers the synthetic data generation pipeline, including client archetype design and anomaly

scenario injection. Chapter 4 details the training and evaluation of the Autoencoder and LSTM models. Chapter 5 presents the streaming pipeline, from event ingestion through enrichment, scoring, and decision-making, including batch integration and the dead letter queue. Chapter 6 covers the MLOps layer: experiment tracking, model promotion, re-training triggers, the supervisor feedback loop, and observability. Chapter 7 describes the Streamlit dashboard and presents the end-to-end validation results. Chapter 8 documents the production enhancements — Kubernetes deployment, CI/CD, secrets management, load testing, and data retention — that would be required to transition the platform from its current development deployment to a production banking environment.

Chapter 1

Context and State of the Art

This chapter presents the institutional context, the regulatory environment governing anti-money laundering in Tunisia, and a review of the anomaly detection techniques that inform the system design.

1.1 Host Institution

Amen Bank is a privately held Tunisian commercial bank operating under the supervision of the Banque Centrale de Tunisie (BCT). As of 2023, the bank operates 150 branches distributed across 9 geographic zones (Tunis I through IV, Cap Bon, Sahel, Sfax, and South), supported by 5 Business Centers and 11 Self-Service Areas. The institution employs 1,139 staff members, of whom 60.5% are assigned to the branch network and 39.5% to head office functions.

The internship was hosted by the **Direction Centrale du Système d’Information** (DCSI), specifically within the **Département des Projets Agence**, which is responsible for developing and maintaining the information systems that support branch-level operations.

The bank’s client base is composed of three economic segments: individuals (61% of deposits), private businesses (33%), and institutional clients (6%). This composition, drawn from the audited 2023 annual report, directly informs the client archetype design described in Chapter 3.

1.2 Branch Teller Operations

The project scope covers four teller-window (*guichet*) operations that constitute the core transactional activity of a retail bank branch:

- **Retrait** (cash withdrawal) — the client receives physical currency from their account. Risk exposure includes identity fraud, unusual withdrawal patterns, and excessive amounts or frequency.

- **Versement** (cash deposit) — the client deposits physical currency into their account. This operation is the primary vector for structuring (*smurfing*): deliberately splitting a large sum into multiple deposits below the reporting threshold.
- **Virement** (bank transfer) — funds are transferred from the client’s account to a beneficiary. Risks include transfers to unknown beneficiaries, duplicate transactions, and amounts inconsistent with the client’s profile.
- **Remise de chèques** (cheque deposit) — the client deposits one or more cheques for collection. Risks include cheques from unknown emitters, amounts inconsistent with client history, and third-party endorsement patterns.

Each operation involves two actors: the **client** initiating the transaction and the **employee** processing it at the teller window. Anomalies can originate from either side — a client structuring deposits to avoid reporting thresholds, or an employee processing an unusual volume of high-value transactions for a concentrated set of clients. This dual-actor nature motivates the independent client and employee profiling described in Chapter 2.

1.3 Regulatory Framework

Tunisian banks operate under a layered anti-money laundering (LAB — *Lutte Anti-Blanchiment*) and counter-terrorism financing (FT — *Financement du Terrorisme*) framework involving three institutional actors.

1.3.1 Banque Centrale de Tunisie (BCT)

The BCT issues binding circulars that define internal control obligations for financial institutions. Circulars 2018-09, 2017-08, and 2021-05 govern LAB/FT internal controls, including the obligation to report transactions exceeding 10,000 DT to the financial intelligence unit. These circulars are restricted documents not publicly available; the system’s regulatory rules are therefore calibrated from the thresholds extractable from CTAF case studies and published legislation rather than a complete BCT circular inventory. This gap is documented explicitly rather than papered over.

1.3.2 Commission Tunisienne des Analyses Financières (CTAF)

The CTAF is Tunisia’s financial intelligence unit, responsible for receiving and analyzing suspicious transaction reports from financial institutions. Its 2021 annual report provides the behavioral ground truth for what suspicious operations look like in the Tunisian context: espèces are involved in 68% of flagged cases, virements in 56%, and chèques in 34%. The concentration of reported amounts around the 10,000 DT threshold — with

85% of flagged operations falling in the 5,000–20,000 DT band — directly informed the near-threshold detection logic in the system’s decision service.

1.3.3 Law n°41-2024

Effective February 2025, this law caps individual cheques at 30,000 DT. BCT data from Q1 2026 confirms a 24.9% collapse in cheque usage volume following the law’s implementation. This regulatory change required adjusting the cheque deposit archetype parameters and introduced a new anomaly scenario: cheque amounts clustering just below the 30,000 DT ceiling, analogous to the structuring pattern observed below the 10,000 DT reporting threshold.

1.4 Existing Detection Capabilities

The Amen Bank 2023 annual report confirms that the bank already operates an AI-based anti-money laundering system. Based on the report’s description, this system performs *bank-wide compliance reporting on transaction streams*: it serves a compliance officer audience, operates at a quarterly reporting tempo, and produces output aligned with CTAF regulatory requirements.

The system developed in this internship occupies a different position in the same regulatory umbrella. It performs *real-time decision support for branch supervisors on teller operations*: it monitors four specific caisse operations, profiles both the client and the employee as independent actors, operates at near-real-time latency, and uses sequence-aware LSTM modeling to detect behavioral drift over time. The two systems are complementary rather than competing — the existing system answers “should the bank file a regulatory report?” while this project answers “should the branch supervisor investigate this transaction now?”

1.5 State of the Art

This section reviews the techniques that inform the system’s design, organized from traditional approaches to deep learning methods and explainability.

1.5.1 Rule-Based Detection

The earliest and still most widely deployed approach to transaction monitoring relies on deterministic rules: fixed thresholds, velocity checks, and pattern matching defined by compliance officers. Bolton and Hand [1] provide a comprehensive survey of statistical

fraud detection methods, noting that rule-based systems dominate production deployments due to their interpretability and auditability.

Rule-based systems have two structural limitations. First, they require manual maintenance — every new fraud typology demands a new rule, and threshold calibration is labor-intensive. Second, they cannot detect novel patterns that fall outside the predefined rule set. Despite these limitations, regulatory requirements often mandate deterministic checks (such as the 10,000 DT reporting threshold) that cannot be delegated to a probabilistic model. The system developed in this project retains deterministic rules in the Decision Service while delegating pattern detection to the deep learning models — combining the auditability of rules with the adaptability of learned representations.

1.5.2 Statistical Anomaly Detection

Statistical methods model the distribution of normal behavior and flag observations that deviate significantly. Two methods are commonly used as baselines in the anomaly detection literature:

Isolation Forest [2] constructs an ensemble of random trees that partition the feature space. Anomalies, being few and different, require fewer partitions to isolate and thus have shorter average path lengths. The method is efficient on high-dimensional data and does not require distributional assumptions, but it treats each observation independently — it cannot model temporal dependencies between successive transactions.

Local Outlier Factor (LOF) [3] computes the local density of each observation relative to its neighbors. Points in low-density regions relative to their neighborhood are flagged as outliers. LOF is effective for detecting local anomalies in heterogeneous datasets but shares the same limitation as Isolation Forest: it operates on individual data points without temporal context.

Both methods serve as useful baselines but are fundamentally point-wise detectors. Detecting behavioral drift — a client whose transaction pattern gradually shifts over weeks — requires models that consume sequences rather than individual events.

1.5.3 Autoencoders for Anomaly Detection

An Autoencoder is a neural network trained to reconstruct its input through a compressed latent representation. When trained exclusively on normal data, the network learns to encode and decode the patterns present in legitimate transactions. Anomalous inputs, which differ from the training distribution, produce high reconstruction errors that serve as anomaly scores.

Sakurada and Yairi [4] demonstrated that Autoencoders outperform linear PCA-based

methods for anomaly detection in multivariate time series, particularly when the normal data occupies a nonlinear manifold. An and Cho [5] formalized the use of reconstruction error as an anomaly score and showed that variational Autoencoders provide calibrated uncertainty estimates alongside the score.

In the financial domain, Autoencoders are well-suited to the class imbalance problem: anomalies represent less than 1% of transactions, making supervised classification impractical without extensive labeled data. By training only on normal transactions, the Autoencoder avoids the need for labeled anomalies entirely — a critical advantage in banking environments where confirmed fraud labels are scarce and often delayed by months.

The system uses an Autoencoder as the *point anomaly detector*: each transaction is scored independently based on how well it can be reconstructed from the learned normal manifold.

1.5.4 LSTM Networks for Sequential Anomaly Detection

Long Short-Term Memory (LSTM) networks, introduced by Hochreiter and Schmidhuber [6], address the vanishing gradient problem that prevents standard recurrent networks from learning long-range dependencies. The LSTM cell’s gating mechanism (input, forget, and output gates) allows the network to selectively retain or discard information across arbitrarily long sequences.

For anomaly detection, the LSTM is trained as a *next-step predictor*: given a sequence of recent transactions, it predicts the features of the next transaction. The deviation between the predicted and observed features serves as the anomaly score. Bontemps et al. [7] applied this approach to network intrusion detection, showing that LSTM-based prediction models detect subtle sequential anomalies that point-wise methods miss.

In the banking context, the LSTM captures behavioral patterns that unfold over time: a client who normally transacts once per month but suddenly executes five transactions in three days, or a gradual increase in transfer amounts to a new beneficiary over several weeks. These sequential patterns are invisible to the Autoencoder, which evaluates each transaction in isolation.

The system uses the LSTM as the *sequential anomaly detector*: it consumes a sliding window of recent transactions per client and flags deviations from the predicted behavioral trajectory.

1.5.5 Score Fusion

When multiple anomaly detectors operate in parallel, their scores must be combined into a single decision. Aggarwal [8] surveys fusion strategies ranging from simple averaging

to learned combination functions. The key challenge is that different detectors may have different score distributions, making naive averaging misleading.

The system uses a sigmoid-weighted fusion mechanism that adapts to account maturity. New accounts with limited transaction history favor the Autoencoder (which needs only a single event) over the LSTM (which needs a populated sequence buffer). As the account matures and the sequence buffer fills, the LSTM’s weight increases. This addresses the cold-start problem inherent in sequential models without requiring a separate cold-start detection pathway.

1.5.6 Explainability with SHAP

Deploying a deep learning model in a regulated environment requires that every alert be accompanied by an explanation that a non-technical supervisor can understand and act upon. Lundberg and Lee introduced SHAP (SHapley Additive exPlanations), which attributes a model’s prediction to individual input features using game-theoretic Shapley values. SHAP provides three properties that make it suitable for this context: local accuracy (the attributions sum to the model output), missingness (features not present contribute zero), and consistency (increasing a feature’s contribution never decreases its attributed importance).

For the Autoencoder, the system uses KernelSHAP — a model-agnostic approximation that treats the reconstruction error function as a black box. DeepSHAP is inapplicable because the anomaly score subtracts the input from the reconstruction, breaking the standard forward-pass tracing that DeepSHAP requires. For the LSTM, SHAP values over the full sequence-length-by-feature-count tensor produce attributions too granular for a supervisor to interpret. The system instead uses an expected-versus-actual comparison: for each interpretable feature, it reports the LSTM’s predicted value alongside the observed value, making deviations immediately legible.

1.5.7 Streaming Architectures for Real-Time Detection

Detecting anomalies in real time requires an architecture that processes events as they arrive rather than in periodic batches. Kreps et al. [10] introduced the log-based streaming paradigm with Apache Kafka, where events are written to an append-only log partitioned by key and consumed by independent services at their own pace. This decouples producers from consumers and enables horizontal scaling of individual processing stages.

The system adopts this paradigm using RedPanda, a Kafka-compatible broker that eliminates the JVM and ZooKeeper dependencies of Apache Kafka while maintaining API compatibility. Four topics form a processing chain: raw events are enriched with behavioral features, scored by the ML models, evaluated by the decision service, and written

to a persistent store for dashboard consumption. Partitioning by client identifier ensures that all transactions for a given client are processed in order — a requirement for the LSTM’s sequential modeling.

1.6 Problem Statement and Project Objectives

The detection of anomalous teller operations at Amen Bank currently relies on manual rules and periodic compliance reviews. This approach cannot scale to the transaction volume of 150 branches, cannot adapt to evolving fraud typologies without manual rule updates, and provides no real-time decision support to branch supervisors.

The objective of this internship is to design and implement a **real-time anomaly detection platform** that:

1. Ingests teller transactions as they occur and processes them through a streaming pipeline with near-real-time latency.
2. Profiles both the client and the employee as independent behavioral actors, detecting anomalies originating from either side.
3. Combines a point anomaly detector (Autoencoder) with a sequential anomaly detector (LSTM) to capture both individual transaction deviations and behavioral drift over time.
4. Accompanies every alert with a human-readable explanation generated by SHAP, enabling supervisors to make informed decisions without machine learning expertise.
5. Provides a supervisor dashboard for alert review, operational monitoring, and transaction simulation.
6. Implements the MLOps infrastructure — experiment tracking, automated retraining, drift detection, and observability — required to maintain the system beyond initial deployment.

Chapter 2

System Architecture and Design

This chapter presents the overall system architecture, the data schemas that define the contract between services, and the key design decisions that shaped the platform. Every choice is grounded in the operational constraints of branch teller banking in the Tunisian regulatory context.

2.1 System Overview

The platform follows a **streaming-first architecture** with four microservices chained via message topics, supported by a hot/cold data layer and a batch pipeline for profile rebuilding and model retraining. The system monitors two independent actors — the **client** initiating the transaction and the **employee** processing it — reflecting the dual-actor nature of branch teller operations where anomalies can originate from either side.

The streaming pipeline processes each transaction through four stages: ingestion of the raw event, enrichment with behavioral context from client and employee profiles, scoring via two complementary deep learning models, and a decision layer that merges the ML signal with deterministic regulatory rules. The output is a tiered alert with a human-readable explanation, consumed by a supervisor dashboard.

Three foundational choices define the architecture:

1. **RedPanda over Apache Kafka.** RedPanda provides a Kafka-compatible API without the JVM runtime or ZooKeeper dependency, reducing operational complexity while maintaining protocol compatibility. The project’s throughput — hundreds of events per minute from 150 branches — does not require Kafka’s horizontal scaling capabilities. Critically, the architecture remains **language-agnostic at topic boundaries**: any service can be rewritten in a JVM-based language without changing the topic contracts.
2. **Dual-actor profiling.** Both the client and the processing employee are independently profiled. Each actor has a hot profile in Redis for real-time lookups and a cold profile in PostgreSQL for historical analysis. This design is moti-

vated by CTAF case studies where collusion between a client and a specific teller was a key pattern (e.g., *passeur de fonds* scenarios where the same employee processes repeated suspicious transactions for the same client).

3. **SHAP co-located in the Scoring Service.** Explainability is computed at scoring time, not at display time. The SHAP explanation travels as part of the event payload through the Decision Service to the `decisions` topic. This simplifies the dashboard to a single-topic consumer and guarantees that every alert carries its explanation, regardless of how it is consumed downstream.

2.2 Streaming Topology

Four RedPanda topics form the streaming backbone:

`raw-events` → `enriched-events` → `scored-events` → `decisions`

All four topics are **partitioned by `client_id`**. This is a deliberate constraint driven by the LSTM model's requirements: the LSTM treats each client's transaction history as a temporal sequence and predicts what the next event should look like. Cross-operation ordering must be preserved per client — a retraits followed by a versement followed by a virement must arrive at the Scoring Service in exactly that order. Partitioning by `client_id` guarantees this ordering within each partition.

A single `raw-events` topic carries all four operation types using a **tagged-union envelope**: a shared set of common fields plus an operation-specific `payload` object. The alternative — four separate topics, one per operation type — was rejected because it would break the cross-operation temporal ordering that the LSTM requires. If retraits events and virement events travel on separate topics, there is no guarantee of relative ordering at the consumer side.

Two additional topics support the pipeline:

- `dlq` (Dead Letter Queue) — receives events that failed processing after retry exhaustion, preserving the original event alongside error metadata for post-mortem analysis.

Between the streaming services, two **sink consumers** persist data to PostgreSQL: one writes raw events to the `transactions` table, the other writes decisions to the `alerts` table and updates risk history counters in Redis. The sinks operate with independent consumer groups, decoupling persistence from the transform-and-forward pipeline.

2.3 Hot/Cold Data Layer

The platform maintains two complementary data stores, each serving a distinct access pattern:

- **Redis (hot layer)** stores data that must be available with sub-millisecond latency during streaming enrichment: client profiles (`profile:client:{id}`), employee profiles (`profile:employee:{id}`), archetype baselines (`baseline:{archetype}`), per-client event buffers for velocity feature computation (`buffer:client:{id}`), and LSTM sequence buffers (`sequence:client:{id}`).
- **PostgreSQL (cold layer)** stores the persistent record of all transactions, alert history, profile snapshots for audit and retraining, and master reference data (clients, employees, accounts). It serves as the source of truth for batch processing and the dashboard’s query backend.

The nightly batch pipeline reads from PostgreSQL, recomputes all aggregate statistics and behavioral distributions for both actors, writes snapshots back to PostgreSQL for audit, and refreshes the Redis hot profiles. This ensures that the hot layer reflects the latest behavioral patterns without requiring the streaming services to maintain complex windowed aggregations.

LSTM sequence buffers in Redis store **unscaled** feature vectors. The Scoring Service applies the current scaler on-the-fly at inference time. This design ensures that a weekly model retrain — which produces a new scaler — does not invalidate the persisted sequences, avoiding a full buffer rebuild on every retrain.

2.4 Raw Event Schema

The raw event uses a tagged-union envelope: a common set of 11 fields shared by all operations, plus a `payload` field containing operation-specific data. Two principles govern the schema:

1. **Raw events contain only facts observable at transaction time.** No aggregate statistics, no derived fields. Statistics belong in profiles; derived features belong in the enriched event.
2. **No lagging fields.** A cheque’s bounce status, for example, is only known after clearing and belongs to a separate downstream event, not the raw event.

Table 2.1: Common envelope fields

Field	Type	Description
<code>event_id</code>	UUID	Unique event identifier
<code>client_id</code>	UUID	Client initiating the transaction
<code>account_id</code>	UUID	Target account
<code>employee_id</code>	UUID	Teller processing the transaction
<code>branch_id</code>	String	Branch where the operation occurs
<code>timestamp</code>	Datetime	UTC timestamp
<code>amount</code>	Float	Transaction amount
<code>currency</code>	String	ISO 4217 code (typically TND)
<code>channel</code>	Enum	<code>guichet</code> , <code>GAB</code> , etc.
<code>operation_type</code>	Enum	<code>retrait</code> , <code>versement</code> , <code>virement</code> , <code>cheque</code>
<code>payload</code>	JSON	Operation-specific data (see below)

Each operation type defines its own payload structure. For example, a `virement` payload includes `beneficiary_id` and `motif`, while a `cheque` payload includes `emitter_id`, `cheque_number`, and a `remise_id` that groups multiple cheques deposited together. The `remise_id` design preserves per-cheque granularity for anomaly scoring while maintaining the grouping context — each cheque generates its own event rather than one event per remise.

2.5 Dual-Actor Profiling

2.5.1 Client Profile

The client profile comprises approximately 194 fields organized across six categories:

1. **Personal information** (7 fields): archetype, maturity status (`cold_start` or `mature`), home branch, client type (individual or business), account opening date.
2. **Transaction statistics** (113 fields): for each combination of operation type (4) and time window (24h, 7d, 30d, 90d), six statistics are maintained: count, sum, mean, standard deviation, minimum, and maximum. This structure enables contrast features at multiple temporal granularities.
3. **Behavioral patterns** (12 structures): five frequency distributions computed at two horizons (30-day and 180-day) — branch visits, transaction hour, day-of-week, operation type mix, and employee interactions. Two relationship

maps track known beneficiaries and known emitters over 365 days. Jensen-Shannon divergence between the 30-day and 180-day distributions captures behavioral drift in a single scalar per dimension.

4. **Financial state** (12 fields): balance statistics, inflow/outflow sums, and net flow at 30-day and 90-day windows.
5. **Risk history** (49 fields): alert counts per tier (INFO, REVIEW, ALERT, BLOCK) per window (30d, 90d, 180d), each split into confirmed, rejected, and pending by the supervisor. This enables recidivist escalation and false-positive dampening in the Decision Service.
6. **LSTM sequence buffer**: the last 50 events for this client, stored as a bounded Redis list. The sequence length matches the training configuration to prevent any train/inference context mismatch.

2.5.2 Employee Profile

The employee profile mirrors the client profile structure with two key differences. First, **financial state is excluded** — monitoring an employee’s personal finances is out of scope and raises ethical concerns. Second, a **peer comparison** category is added, comprising five z-scores at two horizons (30-day and 90-day):

- Volume z-score (transactions processed relative to peers)
- Error rate z-score
- Near-threshold ratio z-score (fraction of transactions in the 8,000–9,999 DT smurfing band)
- Client concentration z-score (serving few vs. many distinct clients)
- Client-location mismatch z-score

A normalization challenge arises from the branch structure: with only 2 tellers per branch, per-branch z-scores are statistically meaningless. The solution uses a **split normalization** approach. Volume metrics are first normalized per-branch (employee volume divided by branch average volume), then z-scored bank-wide. This prevents high-volume branches from appearing as outliers while still detecting employees who handle disproportionate volume within their branch. Ratio metrics (near-threshold ratio, error rate, client concentration) are z-scored directly bank-wide, since ratios are already volume-independent.

2.6 Enriched Event Design

The enriched event is the feature vector consumed by the Scoring Service. Its design underwent a fundamental revision during the project that produced the single most impactful improvement in detection performance.

2.6.1 The Profile Dominance Problem

The initial enriched event schema contained 74 features, many of which were raw profile field dumps — weekly snapshot means, standard deviations, and full distribution vectors repeated identically across every event from the same client within the same week. The per-event signal (how unusual *this* transaction is) was drowned out by profile noise (what the client’s long-term averages are). Both the Autoencoder and the LSTM failed to distinguish normal from anomalous events, producing AUC scores near random (~ 0.57).

2.6.2 Contrast Feature Redesign

The solution was to replace profile dumps with **contrast features**: each feature measures the deviation of the current event against the client’s established baseline. The feature count dropped from 74 to 30, organized as follows:

Table 2.2: Enriched event feature schema (30 features)

Category	Count	Features
Raw event	7	amount, hour, 4 operation one-hot flags, is_home_branch
Amount contrast	6	z_amount, amount_to_mean_ratio, is_above_threshold, is_near_threshold, is_round_amount, is_near_cheque_ceiling
Operation contrast	1	op_type_probability (from 30d distribution)
Timing contrast	2	is_outside_hours, day_distance_from_preferred
Velocity	8	tx_count_24h, tx_count_7d, cumulative_amount_24h, has_duplicate_recent, near_threshold_count_7d, employee_tx_count_24h, same_employee_client_count_24h, is_new_counterparty
Context	6	5 archetype one-hot flags, account_age_days

The key invariant: the `_compute_features()` function that produces these 30 features is a **pure function** — identical inputs always produce identical outputs, with no I/O or side effects. Both the streaming pipeline and the batch training pipeline call this same function, guaranteeing zero feature drift between training and inference. This is the single most important technical invariant in the system.

2.7 Scoring Architecture

The Scoring Service hosts two complementary models and fuses their outputs into a single anomaly score.

2.7.1 Dual-Model Rationale

The **Autoencoder** is trained on normal transactions only and flags events with high reconstruction error. It excels at **point anomalies** — single transactions that are unusual

in isolation (e.g., a 15,000 DT retrain from a salaried client who averages 300 DT). It works from the very first event, requiring no transaction history.

The **LSTM** treats a client’s transaction history as a temporal sequence and predicts what the next event should look like. It excels at **sequential anomalies** — patterns that are only suspicious in context (e.g., a gradual increase in versement frequency that individually looks normal but collectively signals structuring). It requires sufficient sequence history to be meaningful.

2.7.2 Score Fusion

The two models produce scores on fundamentally different scales (MSE reconstruction error for the AE, prediction error for the LSTM). Three mechanisms normalize and combine them:

1. **Percentile normalization.** Each model’s raw score is mapped to a percentile rank against a reference distribution computed from normal training data. This places both scores on a common $[0, 1]$ scale where 0.95 means “higher than 95% of normal events.”
2. **Maturity-adaptive weighting.** A sigmoid function of `days_since_account_opening` (midpoint 90 days) controls the weight between models. New accounts favor the Autoencoder; mature accounts favor the LSTM. `days_since_account_opening` was chosen over event count because event count penalizes low-frequency archetypes — a retiree who transacts twice a month is not “new,” but has a low event count.
3. **Weighted combination:** $S_{\text{fused}} = w_{\text{lstm}} \cdot S_{\text{lstm}} + (1 - w_{\text{lstm}}) \cdot S_{\text{ae}}$

2.7.3 Cold-Start Scoring

Clients with fewer than 5 events in their sequence buffer fall outside the training distribution entirely (the training pipeline applies a warm-up filter). A three-tier progression handles this:

Table 2.3: Cold-start scoring progression

Buffer length	Scoring behavior	Tier cap
0–4 events	Neutral score (0.5), <code>cold_start=True</code>	REVIEW max
5–9 events	AE-only ($w_{\text{lstm}} = 0$)	Full range
10+ events	Full AE + LSTM fusion	Full range

Cold-start events still pass through all regulatory rules — a 15,000 DT transaction must trigger the BCT reporting flag regardless of buffer length — but the ML-derived tier is capped at REVIEW to prevent score saturation from generating false BLOCKs.

2.7.4 Conditional SHAP

SHAP explanation is computationally expensive ($\sim 100\text{ms}$ per event via KernelExplainer). Since approximately 95% of events score below the REVIEW threshold, SHAP computation is gated behind the threshold check. Events below REVIEW take the fast path ($\sim 5\text{ms}$); events above receive full SHAP explanations. The Autoencoder uses KernelExplainer (DeepExplainer was ruled out because the anomaly score involves a subtraction outside the model’s forward pass). The LSTM uses an expected-versus-actual comparison: the top-3 features with the largest deviation between the predicted and observed values, filtered through a whitelist of interpretable continuous features.

A deterministic template dictionary maps feature names to parameterized natural language sentences, producing consistent and auditable explanations. An LLM-based approach was rejected due to bank security constraints and non-deterministic outputs. Example: “*This versement is suspect because the amount (8,450 DT) is $4.2\times$ the client’s average, and it is the 3rd deposit in 48 hours.*”

2.8 Decision Service

The Decision Service is a stateless three-layer engine that merges the ML signal with deterministic regulatory rules and historical context.

2.8.1 Layer 1 — Score Thresholds

Three thresholds calibrated on the validation set define the alert tiers:

Table 2.4: Alert tier thresholds

Tier	Threshold	Meaning
REVIEW	$T_1 = 0.95$	Supervisor reviews within 24 hours
ALERT	$T_2 = 0.99$	Immediate supervisor attention required
BLOCK	$T_3 = 0.999$	Transaction held, requires override

Events below T_1 receive the INFO tier: logged for audit but requiring no action.

2.8.2 Layer 2 — Regulatory Rule Escalation

Eight rules can **escalate** the tier determined by Layer 1 but **never de-escalate** it. This is a hard invariant: a regulatory flag cannot reduce alert severity. All regulatory flags are logged for audit regardless of whether they trigger an escalation.

Table 2.5: Regulatory escalation rules

#	Rule	Condition	Escalation
1	BCT reporting	Amount $\geq$ 10,000 DT <i>and</i> $z_{\text{amount}} \geq 2$	→ ALERT
2	Smurfing band	Amount in 8,000–9,999 DT <i>and</i> ≥ 2 near-threshold txns in 7d	→ ALERT
3	Cheque ceiling	Amount in 25,000–30,000 DT (Law n°41-2024)	→ REVIEW
4	Passeur de fonds	Occasional client, ≥ 5 versements in 24h	→ ALERT
5	New beneficiary	Virement $\geq$ 5,000 DT to first-time beneficiary	→ REVIEW
6	Duplicate	Matching operation within time window	→ REVIEW
7	Round amount	Amount $\geq$ 5,000 DT, round, with high 24h cumulative	→ REVIEW
8	Frequency burst	$\geq 5 \times$ expected daily rate	→ REVIEW

Rule 1 includes a $z_{\text{amount}} \geq 2$ guard: a 15,000 DT virement from a big business client whose average is 20,000 DT is routine and should not flood the REVIEW queue. The guard ensures that only transactions that are *both* above the regulatory threshold *and* unusual for the specific client trigger escalation.

2.8.3 Layer 3 — Risk History Adjustment

The third layer uses the client’s alert history from their profile:

- **Recidivist escalation:** clients with ≥ 3 confirmed alerts in 30 days receive a tier boost — a borderline event from a repeat offender is treated more seriously.
- **Cold-start conservatism:** new accounts (maturity status `cold_start`) receive a more aggressive scoring posture, reflecting the higher risk associated with unestablished behavioral baselines.

2.8.4 Explanation Assembly

The Decision Service produces different explanation structures depending on the alert trigger:

- **ML-only:** fused score crossed threshold → AE SHAP contributions + LSTM expected-vs-actual deviations.
- **Rule-only:** regulatory rule escalated tier → rule description with specific values.
- **Both:** score set initial tier, rule escalated further → combined explanation from all mechanisms.

2.9 Software Architecture

A mid-project requirement — adding a synchronous REST endpoint for the dashboard’s simulation mode — exposed a coupling problem: all business logic was embedded inside Kafka consumer/producer wrappers. A synchronous `/score` request cannot route through the asynchronous streaming pipeline without unacceptable complexity.

Three options were evaluated:

Table 2.6: Synchronous endpoint design evaluation

Option	Description	Verdict
Request-reply via Kafka	Push to topic, correlate response	Rejected (latency)
Bypass Kafka	Duplicate logic or call services	Rejected (duplication)
BFF / Orchestration	Import cores, call in-process	Selected

The solution applied a **core extraction** pattern: each service was split into a core module and a thin streaming wrapper.

- **Core module** (`EnrichmentCore`, `ScoringCore`, `DecisionCore`): receives external dependencies (Redis client, model objects, metrics collector) via **dependency injection**. Exposes a single public method (façade pattern). Contains all business logic. Has no Kafka dependency.
- **Streaming wrapper:** thin Kafka consumer/producer plumbing. Receives a core instance via DI. The `run()` poll loop calls the core’s public method and produces the result to the output topic. Contains no business logic.

The FastAPI simulation API imports the three core modules directly, wires them with shared Redis and model dependencies at startup via a lifespan context manager, and

exposes a synchronous `/score` endpoint that calls `enrich` $\rightarrow$ `score` $\rightarrow$ `decide` in-process. A `dry_run` flag on the enrichment step prevents simulation requests from polluting the streaming pipeline's Redis buffer state.

This pattern also enables the batch enrichment pipeline to call the same `_compute_features()` function, guaranteeing feature parity across three execution contexts: streaming, batch, and simulation.

Chapter 3

Synthetic Data Engineering

This chapter describes the construction of the synthetic training dataset, from regulatory source extraction through archetype design to a fully operational generator producing labeled transactions. In the absence of real transaction data — a common constraint in banking anomaly detection projects — the quality of the synthetic data determines the ceiling of model performance.

3.1 Source Extraction Methodology

The synthetic generator requires two classes of calibration data: parameters describing the *normal population* (how legitimate clients transact) and patterns describing the *flagged population* (what suspicious activity looks like). No single source covers both, so a multi-source extraction strategy was designed.

A structured extraction template was developed with seven trust dimensions — recency, authority, peer review, information volume, geographic relevance, data type, and reproducibility — each scored as Strong, Medium, or Weak. A nine-dimension coverage matrix evaluates each source’s contribution to the generator’s needs: amount distribution, operation type mix, transaction frequency, inter-event time, temporal patterns, spatial/branch patterns, channel mix, beneficiary/emitter known-rate, and operation sequence patterns. The last dimension was added specifically for the LSTM, which requires cross-operation temporal ordering to learn sequential anomalies.

Three sources were extracted using this template. A fourth (BCT circulars on internal LAB/FT control) was identified but confirmed as restricted documents not publicly available, documented as a known gap.

3.2 Source Extractions

3.2.1 S1 — PaySim

PaySim is a synthetic mobile-money simulator based on real transaction logs from an undisclosed African mobile-money provider. Its trust profile is 2 Strong (peer review, reproducibility), 4 Medium, and 1 Weak (geographic relevance — mobile money in sub-Saharan Africa differs structurally from branch teller banking in Tunisia).

PaySim’s primary role is as a **beneficiary graph structure** source: the dataset’s agent-to-agent transfer network provides the best available model for counterparty relationship patterns. Its secondary role is amount distribution shape and day-of-month seasonality calibration. Operation mapping is direct for three of four operations (**CASH_OUT** ↔ retrait, **CASH_IN** ↔ versement, **TRANSFER** ↔ virement). The largest gap is **cheque**, which has no PaySim equivalent — check-based operations are absent from mobile-money ecosystems entirely.

The PaySim anomaly base rate of approximately 0.13% was retained as the initial calibration anchor for the generator’s anomaly injection rate.

3.2.2 S2 — Amen Bank 2023 Annual Report

The Amen Bank annual report provides **normal population calibration** — who the clients are and what the bank’s infrastructure looks like. The document has an internal two-tier trust structure: Tier 1 comprises audited financial statements (signed by BDO Tunisie and La Générale d’Audit et Conseil), while Tier 2 covers unaudited management narrative. This distinction guided which figures were treated as hard constraints versus directional indicators.

Key extractions:

- **Client deposit composition:** 61% individuals, 33% private businesses, 6% institutional. This directly calibrates the five archetype population proportions.
- **Branch network:** 150 branches across 9 geographic zones, establishing the branch pool size and zone structure.
- **Employee structure:** 1,139 total employees, 60.5% network-assigned. Combined with the 150-branch figure, this yields the approximation of 2 tellers per branch used throughout the system.
- **Existing AI-AML system:** The report confirms Amen Bank already operates an AI-based AML system for bank-wide compliance reporting. This

validates the project’s positioning as a complementary system, not a replacement.

3.2.3 S3 — CTAF 2021 Annual Report

The CTAF (Commission Tunisienne des Analyses Financières) annual report provides **flagged population calibration** — what suspicious operations look like in the Tunisian regulatory context. Four typological case studies were extracted using a structured 7-field template:

- **Case 1** (espèces-heavy structuring): Multiple versements just below the 10,000 DT reporting threshold, dispersed across branches. Informed the near-threshold ratio feature and the smurfing anomaly scenario design.
- **Case 2** (virement chain): Series of virements to unknown beneficiaries with round amounts. Informed first-time beneficiary detection and round-amount flagging logic in the Decision Service.
- **Case 3** (PPE exploitation): Politically exposed person used as a fund conduit. Validated the design decision to treat PPE as a structural flag layered on archetypes rather than a separate archetype.
- **Case 4** (remise de chèques laundering): Third-party cheques deposited in rapid succession. Informed cumulative amount tracking and cheque-specific anomaly patterns.

Six quantitative structural targets were also extracted: instrument co-occurrence rates (espèces 68%, virement 56%, chèque 34%), BBE (Billets de Banque Étrangers) concentration around the 10,000 DT threshold (85% of operations in the 5,000–20,000 DT band), and predicate offence taxonomy (contrebande 20%, corruption 19%, escroquerie 11%).

An important calibration distinction: the Amen Bank report describes the normal population (>99% legitimate operations), while the CTAF report describes the flagged population (<1% that should trigger alerts). The synthetic generator requires both distributions.

3.2.4 Known Gap — BCT Circulars

BCT (Banque Centrale de Tunisie) circulars 2018-09, 2017-08, and 2021-05 would have provided exhaustive regulatory threshold enumeration for internal LAB/FT controls. These documents are restricted and not publicly available. The gap is documented rather than papered over: the Decision Service’s regulatory rules are based on the thresholds extractable from the CTAF case studies and Law n°41-2024, not a complete BCT circular inventory.

3.3 Client Archetype Design

Five client archetypes were designed across six behavioral dimensions, calibrated using the deposit composition data from S2 (Amen Bank annual report):

Table 3.1: Client archetypes

Archetype	Population	Dominant Op.	Monthly Vol.	Amount Range (DT)
Salaried	49%	Retrait (50%)	~5.7	200–500
Small Business	33%	Versement (55%)	~37.5	500–2,000
Student	5%	Retrait (95%)	~7.2	40–80
Retiree	7%	Retrait/Virement	~3.2	200–500
Big Business	6%	Virement (78%)	~63	2,000–20,000

Population proportions are derived from the 61% individual / 33% private business / 6% institutional split, with the individual segment subdivided approximately 80% salaried, 12% retiree, 8% student. The six behavioral dimensions per archetype are: operation mix (probability distribution over four operations), transaction frequency (mean and standard deviation per operation per month), timing (preferred day of month), amount ranges (mean and standard deviation per operation), counterparty loyalty (probability of transacting with a known beneficiary/emitter), and branch loyalty (probability of using the home branch).

All archetype parameters are externalized in a YAML configuration file, enabling recalibration without code changes.

3.3.1 Regulatory Discovery — Law n°41-2024

During archetype calibration, a regulatory discovery was made: Law n°41-2024 (effective February 2025) caps individual cheques at 30,000 DT. BCT Q1 2026 data confirms a 24.9% collapse in cheque usage volume following this law. This required revising the big business archetype: remise de chèques frequency dropped from approximately 12–15 per month to 2–4, with virement absorbing the difference. A new anomaly scenario — cheque structuring just under the 30,000 DT ceiling — was added to the generator (see Section 3.5).

3.4 Generator Architecture

The generator follows a **4-class, 2-phase** design that separates the behavioral model from the anomaly injection mechanism:

1. **Archetype** (Layer 1): loads behavioral parameters from the YAML configuration file. Defines population-level priors — what a “typical salaried client” looks like on average.
2. **Client** (Layer 2): samples a fixed personality from the archetype prior via Gaussian sampling with clipping constraints. Each client’s parameters (personal amount mean, personal frequency, personal preferred day) are drawn once at creation and remain fixed for the client’s lifetime. Operation mix probabilities are sampled then normalized to sum to 1.
3. **Generator** (orchestrator): executes in two phases. **Phase 1 (planning)** creates the client population, estimates total event volume, and assigns anomaly scenarios to selected clients with specific time windows and difficulty tiers. **Phase 2 (generation)** ticks through each simulated day, determines which clients transact (using a bell curve around their preferred day), generates normal or anomalous events depending on whether the client is in an active anomaly window, and produces the final chronologically ordered dataset.
4. **Event** (inline): per-event noise application, payload construction based on operation type, branch selection respecting branch loyalty, and employee assignment from the visited branch’s pool.

The two-phase approach is essential for controlled anomaly injection. Hard-tier scenarios like smurfing span 21 days and require sufficient normal history before the anomaly window for the LSTM to have learned the client’s baseline. Planning the anomaly windows before generation ensures this temporal constraint is respected.

3.4.1 Branch-Linked Employee Assignment

A critical data quality requirement: each employee is assigned to exactly one branch (2 employees per branch, 300 total across 150 branches). The initial implementation assigned employees from a global pool, allowing the same employee to appear at multiple branches — a physical impossibility in branch banking. This was identified as a bug that contaminated the `repeated_client_employee` anomaly scenario: the scenario requires a specific client to visit a specific teller at a specific branch repeatedly, which is meaningless if the teller can appear anywhere. After fixing employee assignment to be branch-linked, the LSTM’s AUC improved from 0.9430 to 0.9866 because branch-linked employees produce cleaner sequential patterns.

3.5 Anomaly Scenario Design

Nine anomaly scenarios are organized across three difficulty tiers, reflecting the detection challenge they present. Difficulty is defined by how many features an anomaly modifies and over what time horizon.

Table 3.2: Anomaly scenarios by difficulty tier

Tier	Scenario	Description	Actor
Easy (1 day)	Amount spike	Single transaction $5\text{--}10\times$ client mean	Client
	Round amount	Large round amount (5,000–20,000 DT)	Client
	Duplicate virement	Matching virement within short window	Client
Medium (5 days)	Frequency burst	Transaction rate $> 3\times$ daily average	Client
	Cumulative threshold	24h/48h cumulative exceeds norm	Client
	Timing anomaly	Transactions outside branch hours	Client
Hard (21 days)	Smurfing	Versements in 8,000–9,800 DT range, multiple branches	Client
	Cheque structuring	Cheques near 30,000 DT ceiling (Law n°41-2024)	Client
	Repeated client-employee	Same client-teller pair ≥ 3 in 24h	Cross

Each scenario was validated for **feature-space detectability**: the injected anomaly must produce a measurable change in at least one of the 30 enriched features. Scenarios that modified only profile-level aggregates without affecting per-event features were removed during a mid-project audit (reduced from 13 to 9 scenarios). For example, the original `behavioral_drift` scenario gradually shifted a client’s operation mix over 21 days, but this change was absorbed by the profile’s 30-day distribution window and produced no detectable per-event signal.

The difficulty distribution follows a pyramid: 10% easy, 30% medium, 60% hard, reflecting the real-world observation that most suspicious activity is designed to evade detection.

3.6 Data Validation

The generator produces four CSV files per run:

Table 3.3: Generator output files

File	Content	Rows
<code>transactions.csv</code>	All simulated events	243,904
<code>clients_master.csv</code>	Client reference data	10,000
<code>accounts.csv</code>	Account reference data	10,000
<code>employees_master.csv</code>	Employee reference data	300

Key validation metrics on the final dataset:

Table 3.4: Dataset validation

Metric	Value
Total events	243,904
Anomaly rate	0.623% (target: $\sim 0.13\%$)
Anomaly scenarios	9 (all represented)
Unique clients	10,000 across 5 archetypes
Employees	300 (2 per branch, branch-linked)
Branches	150
Cross-branch employee violations	0
Simulation period	180 days (January–June 2025)
Salaried retraits mean	~ 300 DT (consistent with Tunisian salary)

The anomaly rate (0.623%) is higher than the 0.13% target from PaySim because multi-day anomaly windows (5 days for medium, 21 days for hard) produce multiple anomalous events per plan entry rather than one. This was accepted as a reasonable trade-off: the rate remains well within the class imbalance range where reconstruction-based and sequence-based models can learn effectively, and the evaluation metrics (AUC-ROC) are threshold-independent.

3.6.1 Oracle Profile Shortcut

For model training, client personality parameters are exported directly from the generator’s in-memory objects to `clients_master.csv`. This provides the exact behavioral baseline needed for contrast feature computation without requiring batch profile computation over the simulated transaction history. The oracle shortcut produces the same 30 enriched features as the production streaming pipeline, ensuring training-inference feature parity. In the production system, these baselines are replaced by batch-computed

profiles from real transaction history — the pipeline handles this substitution without code changes.

Chapter 4

Model Training and Evaluation

This chapter covers the machine learning pipeline: data preparation, model architectures, the diagnostic process that identified a fundamental feature engineering flaw, and the final model performance after the pipeline rebuild.

4.1 Data Preparation

4.1.1 Temporal Split

A chronological split preserves the temporal ordering that the LSTM relies on. A random split would leak future events into the training set, producing artificially inflated validation metrics. The 180-day simulation period is divided as follows:

Table 4.1: Data split strategy

Split	Day Range	Events	Anomalies
Warm-up (discarded)	1–21	~28,000	—
Train (normal only)	22–126	142,254	0
Validation	127–153	36,548	312
Test	154–180	36,059	175

The first 21 days are discarded as a warm-up period. During this window, client buffers are still populating and velocity features (`tx_count_24h`, `cumulative_amount_24h`) are unreliable. The Autoencoder training set contains **normal events only** — anomalous events during the training window are excluded. This is the standard approach for reconstruction-based anomaly detection: the model learns what “normal” looks like, and any event it cannot reconstruct well is flagged as anomalous.

A `StandardScaler` is fitted on the normal training data only and applied to all splits. The scaler is saved alongside the model for use during inference.

4.1.2 Class Imbalance

With an anomaly rate of 0.623%, standard accuracy metrics are misleading (a model that predicts “normal” for every event achieves 99.4% accuracy). AUC-ROC was chosen as the primary metric because it evaluates model performance across all possible thresholds, independent of the class distribution. Optuna hyperparameter searches use AUC-ROC as the objective function.

4.2 Autoencoder

4.2.1 Architecture

The Autoencoder is a symmetric feedforward network that compresses the input feature vector into a low-dimensional bottleneck and reconstructs it:

$$30 \xrightarrow{\text{encoder}} 20 \rightarrow 10 \xrightarrow{\text{decoder}} 20 \rightarrow 30 \quad (4.1)$$

The bottleneck dimension of 10 (one-third of input) provides enough capacity to represent the structure of normal behavior without allowing the network to learn an identity function. ReLU activations are used between layers. The model is trained with Adam optimizer (learning rate 10^{-3}), batch size 256, for 50 epochs with the best model selected by validation AUC.

4.2.2 Anomaly Scoring

The anomaly score is the **mean squared reconstruction error** between the input vector $\mathbf{x}$ and its reconstruction $\hat{\mathbf{x}}$:

$$s_{\text{ae}} = \frac{1}{d} \sum_{i=1}^d (x_i - \hat{x}_i)^2 \quad (4.2)$$

Normal events, which the model has learned to reconstruct accurately, produce low scores. Anomalous events, which deviate from the learned normal distribution, produce high reconstruction errors.

4.3 LSTM

4.3.1 Architecture

The LSTM treats each client’s transaction history as a temporal sequence. It takes the last 10 enriched events as input and predicts what the next event should look like:

$$\text{LSTM}(\text{input} = 30, \text{hidden} = 64, \text{layers} = 2) \rightarrow \text{Linear}(64, 30) \quad (4.3)$$

The input at each time step is a 30-dimensional feature vector (the enriched event). Two stacked LSTM layers process the sequence, and the final hidden state feeds into a linear projection that predicts the 30 features of the next event. The sequence length of 10 was chosen as a balance between capturing sufficient behavioral context and keeping inference latency low.

4.3.2 Anomaly Scoring

The anomaly score is the mean squared prediction error between the LSTM’s predicted next event $\hat{\mathbf{y}}$ and the actual observed event $\mathbf{y}$:

$$s_{\text{lstm}} = \frac{1}{d} \sum_{i=1}^d (y_i - \hat{y}_i)^2 \quad (4.4)$$

An event that matches the LSTM’s prediction (consistent with the client’s behavioral history) produces a low score. An event that deviates from the predicted pattern produces a high score. This approach is particularly effective for detecting sequential anomalies — patterns that appear normal in isolation but are unusual given the client’s recent activity.

4.3.3 Sequence Construction

Sequences are constructed per client using a sliding window: for each event at position t in a client’s chronologically ordered history, the input is events $[t-10, \dots, t-1]$ and the target is event t . Clients with fewer than 11 events are excluded from LSTM training. In the streaming pipeline, the sequence buffer is maintained in Redis and updated after each event.

4.4 Initial Training Failure and Root Cause Analysis

The initial training attempt used the 74-feature enriched event schema. Three Autoencoder configurations were evaluated via Optuna (71 features with mean MSE, 34 behav-

ioral features, 71 features with max/top-k scoring). The best achieved AUC 0.577. The LSTM with multi-task prediction heads achieved AUC 0.564. Both results were near random, triggering a root cause investigation.

4.4.1 Diagnosis

Two independent root causes were identified:

1. **Undetectable scenarios.** Several of the original 13 anomaly scenarios had stub implementations that did not produce measurable changes in the enriched feature space. For instance, `timing_anomaly` shifted the event hour, but the enriched event contained no per-event hour feature — only a profile-level hour distribution that absorbs single-event changes without producing a detectable signal.
2. **Profile dominance.** The 74-feature enriched event was dominated by raw profile field dumps — weekly snapshot means, standard deviations, and full distribution vectors that were identical across every event from the same client within the same week. The per-event signal was drowned out by profile noise. Anomalous events looked nearly identical to normal events from the same client because both carried the same profile values.

4.4.2 Resolution

Three components were redesigned simultaneously:

- **Generator:** anomaly scenarios reduced from 13 to 9, with each remaining scenario validated for feature-space detectability (see Section 3.5).
- **Enrichment:** complete redesign from profile dumps to contrast features. Feature count dropped from 74 to 30, with every feature measuring the deviation of the current event against the client’s baseline (see Section 2.6).
- **Training data:** oracle profile shortcut introduced, providing exact client baselines without weekly granularity artifacts.

This was the single most impactful change in the project. The improvement was entirely structural — the same model architectures and hyperparameters that produced AUC 0.57 on the 74-feature schema produced AUC 0.98+ on the 30-feature contrast schema.

4.5 Post-Rebuild Results

After the pipeline rebuild, both models were retrained on the redesigned 30-feature schema with branch-linked employees:

Table 4.2: Model performance comparison

Model	Initial AUC (74 feat.)	Rebuilt AUC (30 feat.)	Final AUC (aligned)
Autoencoder	0.577	0.9853	0.9714
LSTM	0.564	0.9866	0.9355

The “rebuilt” column reflects training on the oracle profile shortcut (used during development). The “final” column reflects retraining on the production-aligned 30-feature schema where both streaming and batch pipelines call the same `_compute_features()` function. The AUC drop from rebuilt to final was expected: the Autoencoder’s reconstruction error is less sensitive to input source differences, while the LSTM lost more because sequence modeling benefits from richer per-step representations.

4.5.1 Per-Scenario Analysis

Table 4.3: Per-scenario detection performance (rebuilt models)

Scenario	Tier	AE AUC	LSTM AUC
Amount spike	Easy	0.9998	0.9999
Round amount	Easy	1.0000	1.0000
Duplicate virement	Easy	0.9980	—
Frequency burst	Medium	0.9983	0.9125
Cumulative threshold	Medium	0.9742	0.8540
Timing anomaly	Medium	0.9710	—
Smurfing	Hard	1.0000	0.9981
Cheque structuring	Hard	1.0000	0.9999
Repeated client-employee	Hard	0.9983	0.9976

An unexpected finding: hard-tier scenarios produced higher AUC than medium-tier scenarios. This inversion occurs because hard scenarios (smurfing, cheque structuring) modify high-signal features like `z_amount` and `is_near_threshold` with large deviations, while medium scenarios (frequency burst, cumulative threshold) produce subtler, multi-event patterns that require more context to detect. Difficulty classification reflects real-world detection challenge (deliberately subtle patterns), not model sensitivity.

4.5.2 Complementary Detection Profiles

The two models exhibit complementary strengths that justify the dual-model architecture:

- The **Autoencoder** excels at point anomalies visible in a single event (amount spike, round amount, threshold proximity). It produces high scores from the very first event, with no sequence history required.
- The **LSTM** excels at sequential patterns (repeated client-employee interactions, behavioral sequences). Its performance on medium-tier scenarios is lower because these patterns span multiple events and require sufficient context.

4.6 Framing for Production Deployment

These AUC values validate the system’s ability to learn behavioral patterns and serve predictions end-to-end. They are **not** a production performance benchmark — the models were trained on synthetic data whose statistical properties approximate but do not replicate real Amen Bank transaction distributions.

The engineering argument is more important than the absolute numbers: the system is designed so that replacing synthetic data with real transactions requires only re-running the training pipeline. The same `_compute_features()` function, the same score fusion mechanism, the same decision rules, and the same retraining infrastructure handle real data without code changes. The pipeline is the deliverable, not the model weights.

4.6.1 Multi-Mechanism Convergence: Smurfing as Case Study

Smurfing illustrates how the three detection mechanisms converge on a single anomaly pattern. A smurfing scenario spans 21 days, but neither the Autoencoder (which sees one event at a time) nor the LSTM (which sees a 10-event window) needs to observe the entire window to detect it.

Each smurfing event arrives at the Scoring Service already enriched with velocity features computed from the rolling Redis buffer: `near_threshold_count_7d` accumulates across the smurfing window, `cumulative_amount_24h` reflects recent activity, and `z_amount` captures the deviation from the client’s baseline. These features carry the behavioral *history* into each event’s feature vector, regardless of the model’s own window size.

The three mechanisms contribute distinct signals:

1. **Autoencoder:** each individual smurfing event is anomalous in isolation — a salaried client whose retraits average is 300 DT suddenly performing 8,500 DT

versements produces a high reconstruction error from `z_amount`, `amount_to_mean_ratio`, and `is_near_threshold` alone.

2. **LSTM:** the 10-event sequence shows escalating velocity features (`near_threshold_count_7d` climbing from 0 to 4, `tx_count_7d` rising) that deviate from the predicted normal trajectory. The LSTM does not need to see the full 21-day window — it detects the *acceleration* within its own context.
3. **Decision Service Rule 2:** fires deterministically when the amount falls in the 8,000–9,999 DT band *and* the client has ≥ 2 near-threshold transactions in 7 days. This rule requires no ML inference and catches structuring regardless of model scores.

This convergence is not accidental — it is a direct consequence of the contrast feature design. Because each enriched event carries both its own signal and the accumulated behavioral context from the rolling buffer, even a stateless model (the Autoencoder) can participate in detecting a multi-day pattern. The LSTM adds temporal sensitivity, and the regulatory rules provide a deterministic safety net.

Chapter 5

Streaming Pipeline and Integration

This chapter describes the runtime streaming pipeline: how events flow from ingestion through enrichment, scoring, and decision in near real time. It covers the messaging topology, the software pattern that enables code reuse across streaming, batch, and API paths, the fault tolerance layer, and the integration points with PostgreSQL and the nightly batch rebuild.

5.1 Streaming Topology

The pipeline is organized as a linear chain of four RedPanda topics, each consumed by a dedicated microservice that produces to the next:

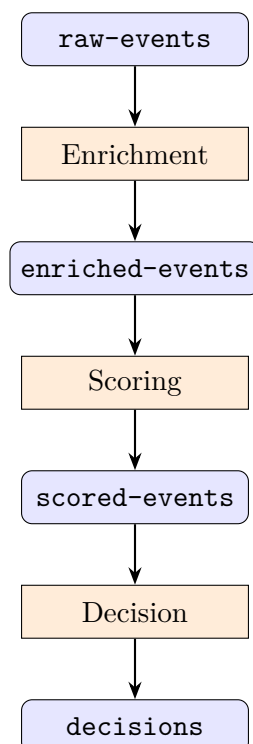


Figure 5.1: Streaming topic chain

All four topics share a single partition key: `client_id`. This is a deliberate constraint. The LSTM model treats each client’s transaction history as a temporal sequence across

all four operation types. If operations were split across separate topics (one per operation type), cross-operation ordering would be lost — a retrain followed by a versegment five minutes later would arrive on different topics with no guaranteed relative ordering. A single topic partitioned by `client_id` guarantees that all events for a given client arrive at the Enrichment Service in the order they occurred, preserving the temporal signal the LSTM depends on.

5.1.1 Broker Choice: RedPanda

RedPanda provides a Kafka-compatible API without the JVM or ZooKeeper dependency. This matters in a Docker Compose deployment where every service competes for memory. A Kafka cluster requires a minimum of three containers (ZooKeeper + broker + schema registry), while RedPanda runs as a single container with the same producer/consumer API. The streaming wrappers use `kafka-python` and are unaware they are communicating with RedPanda rather than Kafka — the switch is transparent at the application level.

5.1.2 Consumer Groups and Offset Management

Each service registers a dedicated consumer group (`enrichment-service`, `scoring-service`, `decision-service`). Consumer groups provide two guarantees that the pipeline relies on. First, offset tracking: if a service restarts, it resumes from the last committed offset rather than replaying the entire topic. Second, horizontal scalability: adding a second instance of the Enrichment Service automatically splits partitions across the two consumers, doubling throughput without code changes. In the current deployment, each service runs as a single consumer, but the architecture supports scaling out by adding replicas.

5.2 Core Extraction Pattern

The system supports three execution paths that all require the same computation:

1. **Streaming** — Kafka consumer loop, one event at a time, writes to Redis buffers
2. **Batch** — Airflow DAG iterating over PostgreSQL rows, no Redis interaction
3. **Simulation API** — FastAPI endpoint, synchronous request-response, read-only Redis access

Duplicating the enrichment, scoring, or decision logic across these three paths would create a maintenance burden and a drift risk: any feature computation change would need to be applied in three places. The solution is a consistent separation between *core logic* and *transport wrappers*.

5.2.1 Core Modules

Each pipeline stage has a core module that encapsulates all domain logic:

Table 5.1: Core modules and their responsibilities

Core	File	Public Method
EnrichmentCore	<code>src/enrichment/core.py</code>	<code>enrich_event()</code>
ScoringCore	<code>src/scoring/core.py</code>	<code>score_event()</code>
DecisionCore	<code>src/decision/core.py</code>	<code>decide_event()</code>

Each core receives its dependencies via constructor injection: a Redis client, model objects, or configuration. The core exposes a single public method (façade pattern) and uses private helpers for sub-steps. Critically, the cores have no knowledge of Kafka, Airflow, or FastAPI — they operate on Python dictionaries and return Python dictionaries.

5.2.2 Thin Streaming Wrappers

The Kafka-specific logic lives in `src/streaming/`, where each wrapper file does exactly three things: poll a topic, call the core’s public method, and produce the result to the next topic. The wrapper receives its core instance via dependency injection at startup. The `__main__` block is the sole wiring point: it creates the Redis client, loads models, constructs the core, injects it into the wrapper, and starts the poll loop.

This separation has a concrete payoff. When the batch pipeline was rewritten (Section 5.9), it imported `EnrichmentCore` and called the same `_compute_features()` method. When the simulation API was built, it imported all three cores and wired them with a shared Redis client. In both cases, zero feature computation code was duplicated.

5.3 Dead Letter Queue and Fault Tolerance

In a streaming pipeline, a single malformed event can block an entire consumer if the error is not handled. The fault tolerance layer classifies failures into two categories and applies different strategies to each.

5.3.1 Failure Classification

Poison pills (`KeyError`, `ValueError`, `JSONDecodeError`) are structurally invalid events that will never succeed regardless of how many times they are retried. These are routed to the DLQ immediately with `retry_count=0`.

Transient failures (`redis.ConnectionError`, `psycopg2.OperationalError`, network timeouts) are infrastructure errors that may resolve on their own. The consumer applies exponential backoff (1s, 2s, 4s) and retries up to 3 times. If all retries fail, the event is routed to the DLQ with `retry_count=3`.

The distinction matters because the two classes require opposite responses. Retrying a poison pill wastes time and will never succeed. Immediately discarding a transient failure loses an event that would have succeeded one second later when Redis reconnected.

5.3.2 ResilientConsumer

The `ResilientConsumer` class (`src/streaming/dlq.py`) wraps all three streaming services. It intercepts exceptions from the core's public method, classifies them using an `isinstance` check against a `TRANSIENT_ERRORS` tuple, applies the appropriate strategy, and produces failed events to a single `dlq` topic.

Each DLQ message is wrapped in a metadata envelope:

Table 5.2: DLQ envelope fields

Field	Description
<code>original_event</code>	The raw event that caused the failure
<code>source_topic</code>	Topic the event was consumed from
<code>service</code>	Service that failed (enrichment, scoring, decision)
<code>error_type</code>	Exception class name
<code>error_message</code>	Exception message string
<code>stacktrace</code>	Full Python traceback
<code>retry_count</code>	Number of retries attempted before routing to DLQ
<code>max_retries</code>	Configured maximum (3 for transient errors)
<code>timestamps</code>	First failure and last failure timestamps

The envelope preserves enough context for an operator to diagnose the failure, replay the event after fixing the root cause, and distinguish between infrastructure outages (transient, clustered in time) and data quality issues (poison pills, scattered across clients).

5.3.3 Structured Logging

The streaming services use structured JSON logging rather than plain `print()` statements. A `JsonFormatter` class (`src/streaming/logger.py`) formats every log line as a JSON object with a fixed schema: `timestamp` (UTC ISO 8601), `service`, `level`, `message`, and optional contextual fields (`event_id`, `client_id`, `error_type`, `retry_count`, `score`, `alert_tier`).

A `setup_logger(service_name)` factory function creates a named logger with the JSON formatter attached. Each streaming service calls this factory once at startup. The structured format enables log aggregation tools to filter, group, and alert on specific fields without parsing free-text messages — for example, filtering all log lines where `error_type = "redis.ConnectionError"` to detect infrastructure degradation.

5.4 Enrichment Service

The Enrichment Service consumes raw events, computes 30 contrast features by comparing the current event against the client’s historical profile, and produces enriched events for the Scoring Service.

5.4.1 Data Sources

For each incoming event, the service fetches two categories of data from Redis:

Table 5.3: Enrichment Service data sources

Source	Redis Key	Content
Client profile	<code>profile:client:{id}</code>	Batch-computed statistics: amount means/stds per operation, operation mix distributions, archetype, account age
Client buffer	<code>buffer:client:{id}</code>	Last 50 raw event summaries (timestamp, amount, operation, payload) for velocity features
Employee buffer	<code>buffer:employee:{id}</code>	Recent events processed by this employee for peer comparison features

The client profile is rebuilt nightly by the batch pipeline (Section 5.9). The buffers are updated per-event by the Enrichment Service itself after feature computation.

5.4.2 Feature Computation

The 30 enriched features are organized into categories that reflect how each feature is derived:

Table 5.4: Enriched feature categories

Category	Count	Source
Raw event (amount, hour, operation one-hot, branch match)	7	Event
Amount contrast (z-score, ratio to mean, threshold flags)	6	Event + Profile
Operation contrast (deviation from expected mix)	1	Event + Profile
Timing contrast (hour deviation, day-of-month deviation)	2	Event + Profile
Counterparty contrast (known vs. new)	1	Event payload
Velocity (24h/7d counts, cumulative amount, duplicates)	5	Redis buffer
Employee (transaction count, client pair count)	2	Redis buffer
Context (archetype one-hot, account age)	6	Profile

The term *contrast feature* is central to the design. Raw event fields (amount, hour, operation type) carry no anomaly signal on their own — a 10,000 DT versement is normal for a big business client but extreme for a student. Every feature is computed relative to the client’s own baseline, transforming absolute values into deviations that are comparable across archetypes.

5.4.3 Amount Statistics Fallback Chain

When a client has no 30-day transaction history for a given operation type (common for infrequent operations like cheque deposits), the amount z-score computation would fail with a division-by-zero. The service applies a three-level fallback chain: 30-day statistics first, then 90-day statistics, then the archetype baseline. This ensures every event receives a meaningful contrast score regardless of the client’s transaction density.

5.4.4 Sequence Buffer Management

After computing the 30 features, the Enrichment Service appends the *unscaled* feature vector to `sequence:client:{id}` in Redis, capped at 10 entries. The Scoring Service later retrieves this sequence, scales it using the training-time `StandardScaler`, and feeds it to the LSTM. The features are stored unscaled because the scaler is owned by the Scoring Service — storing pre-scaled features would create a coupling between the Enrichment Service and the scaler version, breaking independence during model retraining.

5.5 Scoring Service

The Scoring Service consumes enriched events, runs both anomaly detection models, and produces a fused anomaly score with optional SHAP explanations.

5.5.1 Score Fusion

The system runs two complementary models on every event:

- **Autoencoder** — computes a reconstruction error that measures how different the current event is from the “normal” patterns learned during training. The error is converted to a percentile rank against the training distribution.
- **LSTM** — evaluates how surprising the current event is given the client’s recent sequence of transactions. The prediction error is similarly converted to a percentile rank.

The two scores are combined using a sigmoid-weighted fusion controlled by account maturity:

$$w_{\text{lstm}} = \sigma\left(\frac{d - 90}{15}\right)$$

where d is the number of days since the account was opened, and σ is the logistic function. For accounts younger than ~ 75 days, the Autoencoder dominates ($w_{\text{lstm}} \approx 0$). For mature accounts ($d > 105$), the LSTM dominates ($w_{\text{lstm}} \approx 1$). The midpoint of 90 days reflects the minimum history needed for the LSTM’s sequence model to produce reliable predictions.

$$\text{fused_score} = (1 - w_{\text{lstm}}) \cdot s_{\text{ae}} + w_{\text{lstm}} \cdot s_{\text{lstm}}$$

5.5.2 Cold-Start Guard

Clients with fewer than 5 events in their sequence buffer present a distinct problem. Their velocity features (`tx_count_24h`, `cumulative_amount_24h`) sit far outside the training distribution, which applied a 21-day warm-up filter. Without a guard, these zero-velocity vectors produce AE reconstruction errors that map to percentiles above 0.99, generating false BLOCK-tier alerts for every new client.

The Scoring Service detects cold-start conditions by checking the sequence buffer length in Redis. When the buffer contains fewer than 5 events, it returns a neutral score of 0.5 with $w_{\text{lstm}} = 0$ and no SHAP explanations. This creates a three-tier progression:

Table 5.5: Scoring behavior by buffer maturity

Buffer Size	Scoring Mode	Rationale
0–4 events	Neutral (0.5)	Insufficient data for meaningful scoring
5–9 events	AE only	Enough for point anomaly, not for sequence
10+ events	Full AE + LSTM	Both models have sufficient context

5.5.3 Conditional SHAP

SHAP explanations are computationally expensive — KernelExplainer for the Autoencoder requires multiple model evaluations per event. To avoid wasting computation on events that will be classified as INFO, the Scoring Service only generates SHAP explanations when the fused score exceeds the first threshold ($T_1 = 0.95$). Events below this threshold are overwhelmingly normal; explanations for them would never reach the supervisor’s screen.

5.6 Decision Service

The Decision Service consumes scored events and produces the final alert decision: an alert tier (INFO, REVIEW, ALERT, or BLOCK) with an explanation combining model scores and regulatory rule triggers.

5.6.1 Alert Tier Assignment

The base tier is derived from the fused anomaly score using three thresholds:

Table 5.6: Score-to-tier mapping

Score Range	Tier	Supervisor Action
< 0.95 (T_1)	INFO	No action required
≥ 0.95 and < 0.99 (T_2)	REVIEW	Manual review recommended
≥ 0.99 and < 0.999 (T_3)	ALERT	Immediate review required
≥ 0.999	BLOCK	Transaction held pending review

5.6.2 Regulatory Rules

After the score-based tier is assigned, 8 regulatory rules are evaluated. Each rule can *escalate* the tier but never *de-escalate* it. This is a deliberate design constraint: if the model assigns ALERT and a rule would suggest REVIEW, the tier stays at ALERT.

Regulatory flags are always logged for audit purposes regardless of whether they change the tier.

The escalation-only principle exists because rules encode hard regulatory requirements (BCT reporting thresholds, LAB/FT structuring patterns) that must never be overridden by a statistical model. A model might assign a low score to a transaction that happens to hit a regulatory threshold; the rule ensures the supervisor still sees it.

Selected rules illustrate the pattern:

- **BCT threshold rule** — any single transaction exceeding 10,000 DT escalates to at least REVIEW, but only when the amount z-score ≥ 2 . The z-score guard prevents routine big-business transactions (where 10,000 DT is normal) from flooding the REVIEW queue.
- **Smurfing detection** — multiple versements in the 8,000–9,999 DT band within 48 hours escalates to ALERT, targeting deliberate structuring below the reporting threshold.
- **Cheque cap rule** — any cheque exceeding 30,000 DT (the cap established by Law n°41-2024, effective February 2025) escalates to BLOCK.

5.6.3 Cold-Start Tier Cap

Events flagged as `cold_start=True` by the Scoring Service still pass through all regulatory rules — a genuinely suspicious transaction from a new client must still trigger the appropriate flags. However, the Decision Service caps the final tier at REVIEW for cold-start events. This prevents the neutral score of 0.5 from being escalated to ALERT or BLOCK by rules alone, which would generate high-priority alerts for clients who simply have not built enough history yet. The cap is logged in the decision reasons so supervisors understand why the tier is limited.

5.7 Sink Consumers

Two sink consumers persist streaming data to PostgreSQL for historical querying, dashboard display, and the retraining feedback loop.

5.7.1 Raw Events Sink

The `raw_events_sink` consumes from the `raw-events` topic and inserts each event into the `transactions` table. This provides the permanent record that the batch pipeline uses for nightly profile rebuilds and that the dashboard uses for transaction history display.

5.7.2 Decisions Sink

The `decisions_sink` consumes from the `decisions` topic and performs two writes per event:

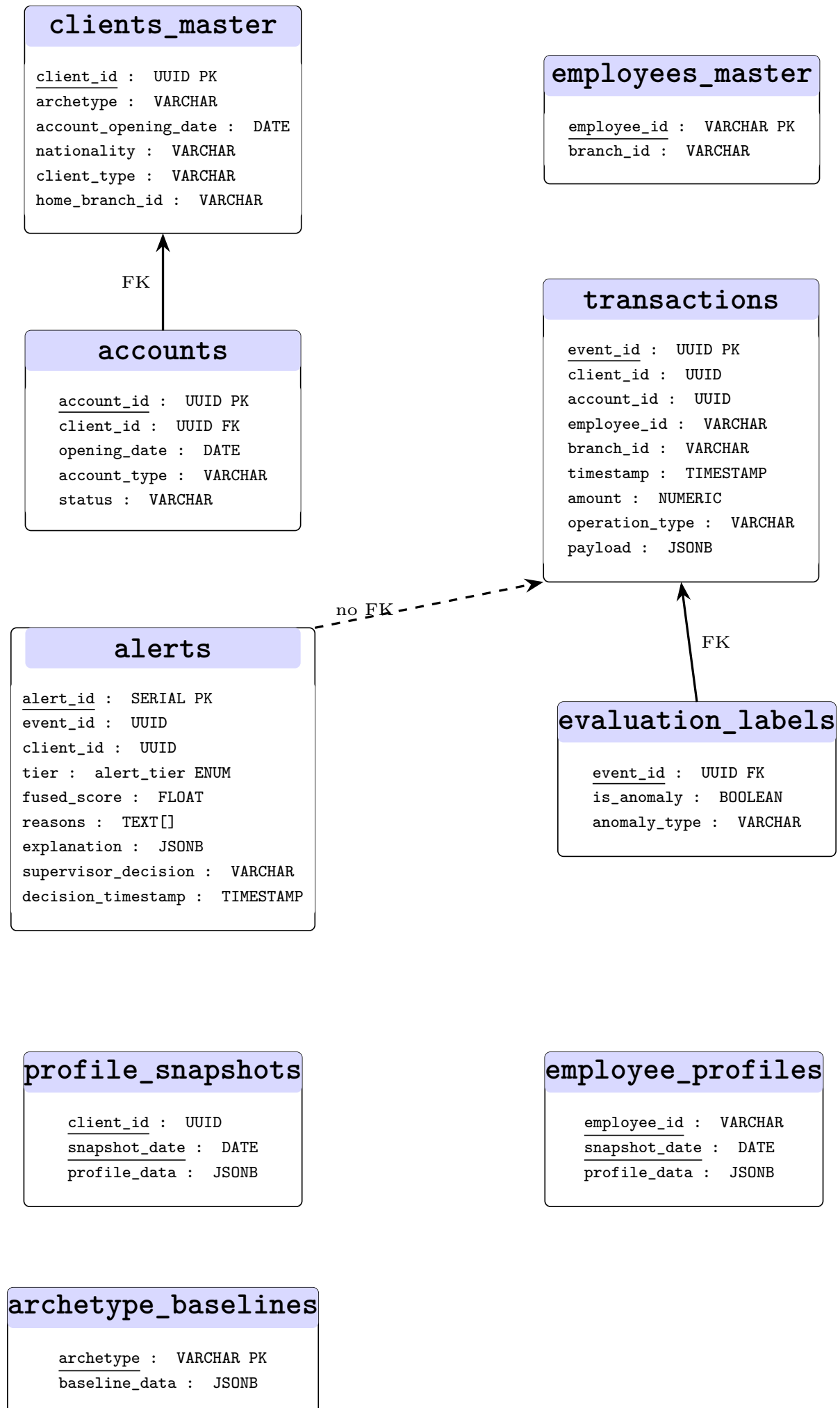
1. **PostgreSQL** — inserts into the `alerts` table with the alert tier, fused score, SHAP explanation (stored as JSONB), and all triggered rule names. The `supervisor_decision` and `decision_timestamp` fields are initially NULL, populated later when the supervisor confirms or rejects the alert through the dashboard.
2. **Redis** — increments risk history counters for the client (`risk:client:{id}`), tracking the count of alerts per tier over rolling windows. These counters feed back into the enrichment features for subsequent events.

5.7.3 Deliberate FK Omission

The `alerts` table does not have a foreign key constraint to the `transactions` table, despite both containing the same `event_id`. This is intentional. The two sink consumers run in parallel: one consumes `raw-events`, the other consumes `decisions`. Because the decision pipeline adds latency (enrichment, scoring, rule evaluation), the decisions sink may attempt to insert an alert before the raw events sink has inserted the corresponding transaction. A foreign key constraint would cause intermittent insertion failures depending on which consumer is faster at any given moment. The `event_id` field still links the two tables logically; the constraint is enforced at the application level rather than the database level.

5.8 Data Persistence Layer

The PostgreSQL database serves as the cold store for the entire system. It receives writes from two streaming sinks and the batch pipeline, and provides reads for the dashboard, the batch profile rebuild, and the retraining pipeline. Figure 5.2 shows the table relationships.



5.8.1 Master Tables

Two master tables store the static reference data loaded once from the generator output:

Table 5.7: `clients_master` — one row per client

Column	Type	Description
<code>client_id</code>	UUID PK	Unique client identifier
<code>archetype</code>	VARCHAR(20)	One of: <code>salaried</code> , <code>small_business</code> , <code>student</code> , <code>retiree</code> , <code>big_business</code>
<code>account_opening_date</code>	DATE	Used to compute <code>account_age_days</code> during enrichment
<code>nationality</code>	VARCHAR(5)	Default TN (Tunisia)
<code>client_type</code>	VARCHAR(30)	Regulatory classification: <code>per-sonne_physique</code> or <code>personne_morale</code>
<code>home_branch_id</code>	VARCHAR(10)	Primary branch for branch-match feature

Table 5.8: `employees_master` — one row per teller

Column	Type	Description
<code>employee_id</code>	VARCHAR(10) PK	Teller identifier (e.g., <code>EMP-0001</code>)
<code>branch_id</code>	VARCHAR(10)	Assigned branch (exactly 2 employees per branch)

5.8.2 Transaction and Event Tables

Table 5.9: `transactions` — populated by the raw events sink

Column	Type	Description
<code>event_id</code>	UUID PK	Unique event identifier
<code>client_id</code>	UUID	Client performing the operation
<code>account_id</code>	UUID	Target account
<code>employee_id</code>	VARCHAR(10)	Teller processing the operation
<code>branch_id</code>	VARCHAR(10)	Branch where the operation occurred
<code>timestamp</code>	TIMESTAMP	Operation timestamp
<code>amount</code>	NUMERIC	Transaction amount in TND
<code>currency</code>	VARCHAR(3)	Always TND
<code>channel</code>	VARCHAR(20)	Always <code>guichet</code> (teller window)
<code>operation_type</code>	VARCHAR(20)	One of: <code>retrait</code> , <code>versement</code> , <code>virement</code> , <code>cheque</code>
<code>payload</code>	JSONB	Operation-specific fields (beneficiary, cheque number, etc.)

Table 5.10: `evaluation_labels` — ground truth for model evaluation

Column	Type	Description
<code>event_id</code>	UUID FK → <code>transactions</code>	Links to the original event
<code>is_anomaly</code>	BOOLEAN	Generator oracle label
<code>anomaly_type</code>	VARCHAR	Scenario name (e.g., <code>smurfing</code> , <code>amount_spike</code>) or <code>NULL</code>

The `evaluation_labels` table is separated from `transactions` by design. Oracle labels are generator metadata used exclusively for model evaluation — they do not exist in a production system with real data. Keeping them in a separate table makes it trivial to exclude them from production deployments without modifying the transaction schema.

5.8.3 Alerts Table

Table 5.11: `alerts` — populated by the decisions sink

Column	Type	Description
<code>alert_id</code>	SERIAL PK	Auto-incrementing alert identifier
<code>event_id</code>	UUID	References the triggering event (no FK — see Section 5.7)
<code>client_id</code>	UUID	Client involved
<code>employee_id</code>	VARCHAR(10)	Teller who processed the operation
<code>branch_id</code>	VARCHAR(10)	Branch location
<code>timestamp</code>	TIMESTAMP	Event timestamp
<code>amount</code>	NUMERIC	Transaction amount
<code>operation_type</code>	VARCHAR(20)	Operation type
<code>tier</code>	<code>alert_tier</code> ENUM	INFO, REVIEW, ALERT, or BLOCK
<code>fused_score</code>	FLOAT	Combined AE + LSTM anomaly score
<code>reasons</code>	TEXT[]	Array of triggered rule names and score-based reasons
<code>regulatory_flags</code>	TEXT[]	Regulatory rules that fired (logged regardless of tier impact)
<code>explanation</code>	JSONB	SHAP explanation with <code>reasons</code> (NLG text) and <code>raw</code> (feature-level detail)
<code>supervisor_decision</code>	VARCHAR(10)	<code>pending</code> , <code>confirmed</code> , or <code>rejected</code>
<code>supervisor_notes</code>	TEXT	Free-text supervisor commentary
<code>decision_timestamp</code>	TIMESTAMP	When the supervisor acted

The `tier` column uses a PostgreSQL ENUM type rather than a VARCHAR. This enforces valid values at the database level and enables direct ordering (`INFO < REVIEW < ALERT < BLOCK`) in queries without a CASE expression.

5.8.4 Profile and Baseline Tables

Three tables support the batch profile rebuild pipeline:

Table 5.12: Profile and baseline tables

Table	Key	Content
profile_snapshots	(client_id, snapshot_date)	Client profile as JSONB: transaction statistics, behavioral distributions, financial state, risk history, recent events buffer
employee_profile_snapshots	(employee_id, snapshot_date)	Employee profile as JSONB: processed transaction stats, client distribution, peer comparison z-scores
archetype_baselines	archetype PK	Per-archetype reference statistics as JSONB: mean amounts, operation mix, frequency norms. Used as the final fallback in the amount statistics fallback chain

Profiles are stored as JSONB rather than as individual columns. A client profile contains approximately 194 fields across 6 categories; normalizing these into columns would produce an unwieldy table that changes shape every time a feature is added. JSONB stores the profile as a single document, and the nightly rebuild overwrites it atomically via `ON CONFLICT` upsert. The tradeoff is that individual fields cannot be indexed or queried efficiently, but the access pattern (load entire profile into Redis) does not require per-field queries.

5.9 Batch Integration

The streaming pipeline depends on client profiles that are too expensive to recompute on every event. A nightly batch pipeline rebuilds these profiles from the full transaction

history in PostgreSQL and loads the results into Redis for the next day's streaming consumption.

5.9.1 Nightly Profile Rebuild

An Airflow DAG (`nightly_profiles`) runs eight tasks with validation gates between stages:

1. Rebuild client profile snapshots from the `transactions` table
2. Validate snapshot completeness (row counts, null checks)
3. Rebuild employee profile snapshots
4. Validate employee snapshots
5. Recompute archetype baselines
6. Validate baselines
7. Hydrate Redis with updated profiles
8. Run a smoke test (score a sample event with the new profiles)

Each task uses `ON CONFLICT` upserts on its target table, making every task individually retrievable. If an Airflow task fails mid-execution and is retried, the upsert ensures that partially-written rows are overwritten rather than duplicated.

5.9.2 Feature Parity Guarantee

The batch profile computation calls the same `EnrichmentCore._compute_features()` method used by the streaming pipeline. The method is a pure function: it takes an event, a profile, and buffer data as arguments and returns 30 features with no side effects. The batch script passes the archetype baseline as a parameter rather than fetching it from Redis, which is the only difference from the streaming path.

This design guarantees that the features used to rebuild profiles overnight are computed with identical logic to the features produced in real time. Any change to the feature computation — adding a feature, modifying a z-score formula, adjusting the fallback chain — is automatically reflected in both paths because both paths call the same function.

5.9.3 Streaming–Batch Coordination

The current deployment does not pause streaming consumers during the batch rebuild window. In practice, the nightly DAG runs during low-traffic hours when transaction volume is minimal, and the Redis hydration step atomically overwrites profile keys. A

brief window exists where a streaming event could read a stale profile (pre-rebuild) and then the next event reads the updated profile, creating a one-event inconsistency. For the current deployment this is acceptable; a production system would implement a coordination mechanism such as pausing consumers during the hydration step or using Redis transactions to swap profile versions atomically. This is documented as future work in Chapter 8.

Chapter 6

MLOps, Observability, and Production Readiness

This chapter covers the infrastructure that surrounds the machine learning models: how experiments are tracked, how models are promoted to production, how the system detects when retraining is needed, and how operational health is monitored. Together, these components close the loop between model serving and model improvement.

6.1 MLOps Lifecycle Overview

A trained model that cannot be retrained, versioned, or monitored is a liability rather than an asset. The MLOps layer transforms the models described in Chapter 4 from static artifacts into a managed lifecycle with four stages:

1. **Train** — execute a training job that produces a candidate model with logged hyperparameters and metrics.
2. **Promote** — compare the candidate against the current production model and replace it only if performance is equal or better.
3. **Serve** — the streaming pipeline loads the promoted model from a shared volume mount, requiring no image rebuild.
4. **Monitor** — observe the score distribution in production and trigger a new training cycle when drift is detected or supervisor feedback indicates degradation.

The lifecycle is largely automated. Training, promotion, serving, and monitoring all run without human intervention. The single human touch point is the supervisor confirming or rejecting alerts in the dashboard (Section 6.6), which feeds one of the three retraining triggers. The promotion gate in particular has no manual approval step between a candidate model and production deployment — a deliberate scope decision discussed in Section 6.3.

Three independent triggers can initiate a new training cycle: a weekly schedule, a drift alert from the monitoring stack, and a supervisor feedback divergence signal. Sections 6.5

and 6.6 describe each path.

6.2 Experiment Tracking with MLflow

MLflow provides the experiment tracking layer. Every training job — whether triggered by schedule, drift, or feedback — logs its artifacts to the same MLflow server, creating a single lineage of all model versions.

6.2.1 Server Configuration

The MLflow server runs as a Docker container with a flat-file backend (`mlruns/` mounted as a volume) rather than a database-backed tracking store. For a single-user development deployment processing one retraining job per week, the flat-file backend avoids the operational overhead of a dedicated metadata database. A production deployment serving multiple concurrent training jobs would migrate to a PostgreSQL-backed store.

6.2.2 Training Job Structure

Each execution of the training script (`src/training/train_all.py`) produces two independent MLflow runs under the experiment `amen-anomaly-detection`: one for the Autoencoder and one for the LSTM. The runs are independent rather than nested because the two models have different architectures, hyperparameters, and evaluation metrics — nesting them would conflate unrelated configuration spaces.

Each run logs:

- **Parameters** — architecture dimensions, learning rate, batch size, number of epochs, sequence length (LSTM only).
- **Metrics** — AUC-ROC on the validation set, computed against the oracle anomaly labels from the synthetic generator.
- **Artifacts** — the trained model file (`.pt`), the fitted scaler, and reference score distributions (`ae_ref_scores.npy`, `lstm_ref_scores.npy`) used by the drift detection system (Section 6.5).

6.3 Model Promotion Gate

After a training job completes, the system must decide whether the newly trained model should replace the one currently serving in production. This decision is handled by an automated promotion gate that compares validation AUC scores.

6.3.1 Promotion Logic

Each model type (Autoencoder and LSTM) is evaluated independently. The training script reads a file `models/production_metrics.json` that stores the AUC of the current production model for each type. If the candidate's AUC is greater than or equal to the production AUC, the candidate is promoted: its `.pt` file, scaler, and reference score distribution are written to the shared model volume, and `production_metrics.json` is updated with the new baseline.

The comparison uses $\geq$ rather than $>$ to allow retraining on updated data to promote a model even when the AUC is numerically identical. In practice, a model retrained on fresher data with the same AUC is preferable to one trained on older data, because the feature distributions it has seen are more representative of current traffic.

On the very first training run — when no production model exists — `production_metrics.json` is absent. The training script detects this and bootstraps by promoting unconditionally, creating the baseline file for all subsequent comparisons.

6.3.2 Scope and Limitations

This gate is sufficient for the current single-user, single-environment deployment. It has two deliberate limitations worth noting:

1. **No human approval step.** A model that passes the AUC threshold is promoted immediately. In a production banking environment, a compliance or risk officer would review model changes before deployment. Adding a manual approval stage — for example, an Airflow task that pauses the DAG until a human confirms via the dashboard — is a straightforward extension.
2. **Single-metric comparison.** The gate evaluates AUC only. A more robust gate would include additional criteria: F1-score at the operating threshold, per-scenario recall to ensure no regression on specific anomaly types, and inference latency to guard against deploying a model that is accurate but too slow for real-time scoring.

6.4 Orchestration with Apache Airflow

Chapter 5 described the nightly profile rebuild DAG (`nightly_profiles`) that maintains the behavioral profiles consumed by the streaming pipeline. This section focuses on the second DAG — `weekly_retrain` — and the infrastructure decisions that apply to both.

6.4.1 Airflow Infrastructure

Airflow runs with a **LocalExecutor**, which executes tasks in the scheduler process rather than distributing them to remote workers. This is sufficient for the current workload of two DAGs with low concurrency. Migrating to **CeleryExecutor** is a configuration change that requires no DAG modifications — only the addition of worker containers and a message broker.

The Airflow metadata tables share the application PostgreSQL instance to conserve memory on the development machine. A dedicated `Dockerfile.airflow` installs only the packages required for orchestration (`requirements-airflow.txt`: `redis`, `numpy`, `pandas`, `psycpg2-binary`, `python-dotenv`, `apache-airflow-providers-docker`) — excluding PyTorch, FastAPI, and SHAP. This keeps the Airflow image approximately 2 GB smaller than the training image.

6.4.2 Weekly Retrain DAG

The `weekly_retrain` DAG runs on a weekly cron schedule and contains three tasks:

1. **Validate data freshness** — confirm that the `transactions` table has received new events since the last training run. If no new data exists, the DAG short-circuits to avoid wasting compute on an identical training set.
2. **Retrain models** — executed via a `DockerOperator` that launches the training image (`amen-anomaly-training:latest`) as a separate container.
3. **Verify promotion** — check that `production_metrics.json` was updated, confirming that at least one model was promoted.

6.4.3 Training Isolation with DockerOperator

The retraining task uses Airflow’s `DockerOperator` rather than a `BashOperator` or `PythonOperator` for two reasons:

- **Dependency isolation.** The training job requires PyTorch, which is large and unnecessary in the Airflow scheduler. Running training in its own container keeps the Airflow image lightweight.
- **Reproducibility.** The training image is pinned as `amen-anomaly-training:latest` with `force_pull=False`, ensuring that the same image is used for scheduled, drift-triggered, and feedback-triggered retraining. The image is rebuilt explicitly when the training code changes, not pulled automatically on every run.

The `DockerOperator` is configured with `mount_tmp_dir=False` to prevent Airflow from

creating a temporary directory mount that conflicts with the container’s working directory. The model volume (`./models`) is mounted into the training container at the same path used by the scoring service, so a promoted model is immediately available to the streaming pipeline on its next restart — no file copy or deployment step required.

6.5 Retraining Triggers

The system supports three independent paths to initiate a retraining cycle. Each path ultimately triggers the same `weekly_retrain` DAG described in Section 6.4, ensuring that all retraining jobs — regardless of their origin — follow the same training, promotion, and artifact-logging pipeline.

6.5.1 Scheduled Retraining

The simplest trigger is a weekly cron schedule configured directly in the Airflow DAG definition. This provides a baseline retraining cadence that ensures the models are periodically refreshed with recent transaction data, even in the absence of any detected anomaly.

6.5.2 Drift-Triggered Retraining

The second trigger detects *score distribution drift* — a sustained shift in the anomaly scores produced by the scoring service that suggests the models are no longer well-calibrated to the current data.

During each training job, the training script saves the distribution of anomaly scores on the validation set as reference arrays (`ae_ref_scores.npy`, `lstm_ref_scores.npy`), logged as MLflow artifacts (Section 6.2). In production, the scoring service exposes the fraction of events scoring above the T1 threshold (0.95) as a Prometheus gauge. A Prometheus alert rule (`config/alert_rules.yml`) fires when this fraction exceeds 15% for a sustained period of one hour:

```
- alert: ScoreDistributionDrift
  expr: score_above_t1_ratio > 0.15
  for: 1h
```

When this alert fires, the detection chain involves four components:

1. **Prometheus** evaluates the alert rule and forwards the firing alert to `AlertManager`.
2. **AlertManager** (port 9093) receives the alert, deduplicates it with a `repeat_interval` of 24 hours to prevent repeated triggers from the same drift episode, and routes it to a webhook receiver.

3. **Webhook relay** (`src/webhook/relay.py`) — a lightweight FastAPI service that translates the AlertManager webhook payload into an Airflow REST API call. This intermediary exists because AlertManager’s native webhook format does not support the HTTP Basic Authentication required by Airflow’s REST API. The relay is a single-endpoint service (Dockerfile built on `python:3.11-slim`, approximately 150 MB) that performs authentication and forwards the trigger with `conf={"trigger": "drift_alert"}`.
4. **Airflow REST API** receives the trigger and starts the `weekly_retrain` DAG. The API is configured with `basic_auth` as the authentication backend.

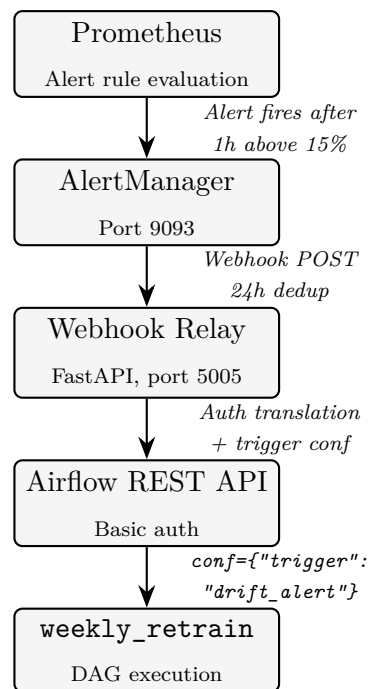


Figure 6.1: Drift-triggered retraining chain. The webhook relay exists because AlertManager cannot natively authenticate against Airflow’s REST API.

The 15% threshold and one-hour window were chosen empirically: under normal operation with the current synthetic data, fewer than 5% of events exceed T1. A sustained elevation above 15% indicates that the score distribution has shifted materially, not that a single burst of legitimate anomalies occurred.

6.5.3 Feedback-Triggered Retraining

The third trigger is described in Section 6.6, as it depends on the supervisor feedback mechanism.

6.6 Supervisor Feedback Loop

The feedback loop is the single human touch point in the MLOps lifecycle. When the streaming pipeline produces an alert at the ALERT or BLOCK tier, the branch supervisor reviews it in the dashboard (Chapter 7) and either confirms or rejects it. This decision is submitted to the simulation API via a `POST /api/v1/feedback` endpoint, which writes the supervisor’s verdict to the `evaluation_labels` table in PostgreSQL with a safe foreign key reference to the original transaction.

These labels serve two purposes. First, they provide ground truth for evaluating model performance on real decisions — a capability that becomes meaningful once the system operates on real transaction data rather than synthetic events with oracle labels. Second, they feed the feedback divergence DAG.

6.6.1 Feedback Divergence DAG

An Airflow DAG (`dags/feedback_monitor.py`) runs daily and executes a SQL query against the `evaluation_labels` table: over a rolling seven-day window, what fraction of ALERT and BLOCK tier events were rejected by supervisors? If the rejection rate exceeds 50% with a minimum of 10 reviewed events, the DAG triggers the `weekly_retrain` DAG with `conf={"trigger": "feedback_divergence"}`.

The 50% threshold reflects a simple principle: if more than half of the system’s high-confidence alerts are being overridden by human reviewers, the models are producing more noise than signal at the upper tiers and should be retrained. The minimum of 10 reviews prevents a single rejected alert early in a deployment from triggering an unnecessary retraining cycle.

6.6.2 Three Triggers, One Pipeline

Table 6.1 summarizes the three retraining paths.

Table 6.1: Retraining trigger comparison

Trigger	Source	Condition	Frequency
Scheduled	Airflow cron	Weekly	Fixed
Drift	Prometheus alert	>15% above T1 for 1h	On demand
Feedback	Supervisor reviews	>50% rejection, ≥ 10 reviews	Daily check

All three paths converge on the same `weekly_retrain` DAG. The `conf` parameter passed at trigger time records which path initiated the run, providing traceability in the Airflow

logs without requiring separate DAG definitions for each trigger type.

6.7 Observability and Monitoring

Observability covers two complementary concerns: *metrics* for real-time dashboarding and alerting, and *logs* for post-incident diagnosis.

6.7.1 Service Metrics

Each streaming service (enrichment, scoring, decision) embeds a `ServiceMetrics` class from `src/monitoring/metrics.py` that exposes two Prometheus instruments:

- A **Counter** tracking the total number of events processed, labeled by service name and outcome (success or error).
- A **Histogram** measuring per-event processing latency in seconds, enabling percentile queries (p50, p95, p99) in Grafana.

Each service exposes its metrics on a dedicated port: enrichment on 9100, scoring on 9101, and decision on 9102. A Kafka exporter container (port 9308) provides broker-level metrics including consumer group lag — the difference between the latest produced offset and the latest committed offset — which serves as the primary indicator of pipeline backpressure. Prometheus scrapes all four targets at a configurable interval (`config/prometheus.yml`).

6.7.2 Failure Domain Separation

A monitoring system must not share a failure domain with the service it monitors. If the enrichment service and its health reporter both depend on Redis, a Redis outage would simultaneously break the service *and* suppress the alert that should report the breakage. This principle ruled out Redis-based heartbeat mechanisms early in the design process.

The chosen architecture avoids this problem: Prometheus pulls metrics over HTTP from each service's embedded metrics endpoint. The only shared dependency is the network layer itself, which is an acceptable residual risk — if the network is down, no monitoring architecture can report it from within the affected network.

6.7.3 Structured Logging

All streaming services use a `JsonFormatter` class (`src/streaming/logger.py`) that outputs one JSON object per log line. Each line includes a timestamp, service name, log level, and message, with optional contextual fields such as `event_id`, `client_id`, and `error_type` attached as structured extras.

A `setup_logger(service_name)` factory configures Python's standard `logging` module with the JSON formatter, replacing all `print()` statements that existed in earlier versions of the codebase. Structured JSON logs enable downstream processing — filtering by service, grouping by error type, or correlating events across services by `event_id` — without fragile regular expression parsing of human-readable log strings.

Chapter 7

Dashboard and Results

This chapter describes the supervisor-facing interface and presents the end-to-end validation results of the complete system.

7.1 Dashboard Architecture

The Streamlit dashboard is the only component the branch supervisor interacts with directly. Every other service — RedPanda, Redis, the scoring pipeline — is invisible infrastructure. The dashboard must therefore surface the right information clearly while remaining responsive under continuous use.

7.1.1 Why the Dashboard Cannot Consume Kafka Directly

Streamlit’s execution model re-runs the entire Python script from top to bottom on every user interaction: clicking a button, changing a filter, or navigating between pages triggers a full re-execution. A Kafka consumer embedded in the script would be destroyed and recreated on each interaction, losing its consumer group offset and replaying messages from the last committed position. This makes direct Kafka consumption architecturally incompatible with Streamlit.

The solution is the sink consumer pattern described in Chapter 5: two independent consumers write streaming results to PostgreSQL (the `alerts` table) and Redis (risk history) continuously. The dashboard reads from PostgreSQL, which provides stable, queryable, filterable access to all historical alerts without the limitations of a streaming consumer.

7.1.2 Data Access Pattern

Reads come directly from PostgreSQL using `psycopg2` with Streamlit’s `@st.cache_data` decorator for query-level caching. Cache TTLs are tuned per page: 10 seconds for the alerts list (where freshness matters) and 30 seconds for KPI aggregations (where stability matters more than immediacy).

A `safe_query()` wrapper in `src/dashboard/db.py` handles connection resilience. If a query fails due to a stale connection — common after Docker restarts or idle timeouts — the wrapper clears the cached connection, establishes a new one, and retries the query transparently. This single-retry pattern avoids the complexity of a connection pool while handling the most common failure mode.

Writes go exclusively through FastAPI endpoints. The dashboard never executes `INSERT` or `UPDATE` statements against the database. This preserves the single-writer principle and ensures that all write-path logic — validation, audit logging, the feedback loop — lives in one place.

7.1.3 Real-Time Refresh

The `streamlit-autorefresh` component triggers a page re-execution every 10 seconds. Streamlit does not support native WebSocket push from server to client, so periodic polling is the pragmatic alternative. The 10-second interval balances responsiveness (a new BLOCK-tier alert appears within 10 seconds of being written to PostgreSQL) against unnecessary database load from overly aggressive polling.

7.1.4 Navigation Structure

The dashboard uses Streamlit’s multi-page application structure, where each page is an independent Python file in a `pages/` directory. Navigating to a page executes only that page’s script, avoiding the overhead of loading data for all pages on every interaction. A shared `components.py` module renders the sidebar on every page, displaying pipeline status indicators and pending alert counts grouped by tier.

7.2 Alerts Page

The alerts page is the supervisor’s primary workspace. It presents a filterable list of alerts with expandable detail for each entry.

7.2.1 Filtering and List View

The supervisor can filter alerts by tier (BLOCK, ALERT, REVIEW), operation type (retrait, versement, virement, cheque), and review status (pending, confirmed, rejected). Each alert row displays a color-coded tier badge, operation type, transaction amount, timestamp, fused anomaly score, and current review status.

7.2.2 Alert Detail

Expanding an alert reveals the full context needed for a supervisory decision: client, employee, and branch identifiers; the triggered rules from the Decision Service with their descriptions; any regulatory flags with warning icons; and the SHAP explanation.

The SHAP explanation is presented in two layers. The first layer displays the human-readable narrative generated by the deterministic NLG templates described in Chapter 4 — sentences such as “This versement is flagged because the amount is 4.2 times the client’s 30-day average and it is the third deposit within 48 hours.” The second layer, hidden behind an expandable section, shows the raw feature-level contributions: per-feature SHAP values for the Autoencoder and expected-versus-actual deviations for the LSTM. This two-layer design serves both audiences: supervisors who need a quick justification and analysts who want to understand the model’s internal reasoning.

7.2.3 Supervisor Feedback

Each pending alert presents two buttons: **Confirm** and **Reject**, with an optional comment field. Clicking either button sends a POST request to the `/api/v1/feedback` endpoint on the FastAPI simulation API, which updates the `supervisor_decision` and `decision_timestamp` columns in the `alerts` table and upserts a corresponding record into the `evaluation_labels` table. The page then clears its cache and re-renders immediately, reflecting the updated status without waiting for the next autorefresh cycle.

7.3 KPIs Page

The KPIs page provides an aggregated view of system activity for operational oversight. Four metric cards at the top display the total alert count and the count for each of the three actionable tiers (BLOCK, ALERT, REVIEW). Below the cards, four Plotly charts visualize different dimensions of the alert data:

- A **pie chart** showing the distribution of alerts across tiers, giving an immediate sense of how frequently the system escalates to high-severity classifications.
- A **bar chart** grouped by operation type, revealing which of the four caisse operations generates the most alerts.
- A **line chart** of alerts per day over the simulation period, useful for identifying temporal patterns or spikes.
- A **bar chart** of the top 20 branches by alert count, highlighting branches that may warrant closer supervisory attention.

All aggregation queries use a 30-second cache TTL, trading a small amount of staleness for reduced database load when multiple supervisors access the page concurrently.

7.4 Simulation Page

The simulation page allows a supervisor to manually construct a hypothetical transaction and submit it for scoring without injecting it into the live pipeline. This serves two purposes: testing whether a suspicious transaction would trigger an alert before it is processed, and training supervisors to understand how the system evaluates different transaction profiles.

The input form collects client ID, employee ID, transaction amount, operation type, and conditional payload fields (beneficiary ID for virements, emitter ID and cheque number for cheques). Client-side validation checks for required fields, positive amounts, and payload consistency before the form is submitted. The form sends a `POST` request to the `/api/v1/score` endpoint, which runs the full scoring pipeline synchronously — enrichment, AE and LSTM inference, score fusion, SHAP explanation, and decision rules — and returns the result.

The response is displayed as a color-coded tier badge, the fused anomaly score, any triggered rules and regulatory flags, and the full SHAP explanation in the same two-layer format used on the alerts page.

7.5 Health Page

The health page queries the Prometheus HTTP API to display the operational status of the streaming pipeline. Six service status indicators show whether each service is reachable: the three core streaming services (enrichment, scoring, decision), the two sink consumers (raw-events sink, decisions sink), and the simulation API. Each indicator queries the Prometheus `up` metric for the corresponding scrape target and displays a green or red badge.

Below the status indicators, the page displays three operational metrics derived from the Prometheus instruments described in Section 6.7: throughput (events processed per second, computed from the Counter's rate), p95 processing latency (from the Histogram), and consumer lag per topic partition (from the Kafka exporter). If Prometheus is unreachable, the page degrades gracefully to a warning message rather than crashing, since a monitoring outage should not prevent the supervisor from accessing other dashboard pages.

7.6 Pipeline Validation and Results

With all components integrated — synthetic data generator, streaming pipeline, batch profiles, ML models, decision rules, sink consumers, and dashboard — the system was validated end-to-end by replaying the full 243,000-event synthetic dataset through the streaming pipeline and observing the resulting alert distribution.

7.6.1 Score Distribution

Table 7.1 shows the tier distribution produced by the validated pipeline after resolving the cold-start and buffer duplication issues described in Chapter 5.

Table 7.1: Alert tier distribution on the synthetic dataset

Tier	Proportion	Interpretation
INFO	89%	Normal operations, no action required
REVIEW	11%	Mild deviations, logged for audit
ALERT	0.35%	Significant anomalies, supervisor review
BLOCK	<0.1%	High-confidence anomalies, immediate action

The score range spans from 0.015 (routine transactions by established clients) to 0.999 (high-confidence anomalies matching multiple detection mechanisms). The distribution is heavily right-skewed, with the vast majority of events scoring well below the T1 threshold of 0.95. This shape is expected: in a functioning anomaly detection system, most transactions should be normal, and the alert tiers should form a steep pyramid with very few events at the top.

7.6.2 Model Performance

Table 7.2 reports the AUC-ROC scores for both models after retraining on the 30-feature schema.

Table 7.2: Model AUC-ROC on the 30-feature schema

Model	AUC (74 features)	AUC (30 features)
Autoencoder	0.9853	0.9714
LSTM	0.9866	0.9355

The AUC decrease after the feature reduction is expected. The 74-feature schema included raw profile statistics that made anomalies trivially separable — a client whose

`tx_count_30d` is zero but who suddenly transacts is obvious to any model. The 30-feature contrast schema forces the models to detect anomalies from relative deviations rather than absolute values, which is harder but more generalizable: the same contrast features work across archetypes without archetype-specific thresholds.

The LSTM loses more than the Autoencoder because it operates on sequences of 30-dimensional feature vectors rather than individual events. Each step in the sequence carries less information per dimension, reducing the LSTM's ability to model fine-grained temporal patterns. Despite this, both models remain well above the 0.90 AUC threshold that would indicate reliable discrimination.

7.6.3 Synthetic Data Caveat

These results validate the system's ability to learn patterns, serve predictions at streaming latency, and produce a realistic alert distribution — but they are not a production performance benchmark. The synthetic data generator produces anomalies according to known scenarios with known parameters. Real-world anomalies are messier, more diverse, and not drawn from the same distribution.

The engineering argument is different from the statistical one: the system is designed so that plugging in real transaction data requires no code changes. The generator is replaced by real event sources; the batch pipeline rebuilds profiles from real history; the models are retrained on real features computed by the same `_compute_features()` function; and the retraining triggers (Chapter 6) ensure that the models adapt as the data distribution evolves. The AUCs reported here confirm that this pipeline works end-to-end, not that it will achieve the same numbers on production data.

Chapter 8

Production Roadmap

This chapter documents the enhancements that would be required to transition the platform from its current Docker Compose deployment to a production-grade banking environment.

8.1 Container Orchestration with Kubernetes

The current deployment runs 20 Docker Compose services on a single host. This is sufficient for development and demonstration, but a production deployment requires horizontal scaling, automatic failover, and resource-aware scheduling across multiple nodes.

Kubernetes would replace Docker Compose as the orchestration layer. Each streaming service (enrichment, scoring, decision) would run as a Deployment with configurable replica counts, allowing the system to scale horizontally under increased transaction volume. Health checks already exposed by the Prometheus metrics endpoints can serve as Kubernetes liveness and readiness probes with no code changes. The model volume mount — currently a shared Docker volume — would migrate to a Persistent Volume Claim, preserving the retrain-without-rebuild pattern established in Chapter 6.

RedPanda, Redis, and PostgreSQL would each run as StatefulSets with persistent storage, or be replaced by managed cloud equivalents depending on the bank’s infrastructure policy.

8.2 Continuous Integration and Deployment

The project currently has no automated CI/CD pipeline. Code changes are tested manually and deployed by rebuilding Docker images. A production pipeline would introduce three stages:

1. **Continuous Integration** — on every commit, run unit tests for the three core modules (EnrichmentCore, ScoringCore, DecisionCore), validate that the feature computation produces the expected 30-dimensional output, and lint the codebase.

2. **Image Build** — on merge to the main branch, build and tag Docker images for each service with the commit hash, ensuring traceability between deployed code and source control.
3. **Staged Deployment** — deploy to a staging environment first, run the E2E test script against the staging pipeline, and promote to production only after validation passes.

The core extraction pattern (Chapter 5) simplifies CI significantly: the pure core modules can be unit-tested without Redis, PostgreSQL, or RedPanda dependencies, covering the majority of the business logic with fast, isolated tests.

8.3 Secrets Management

Database credentials, API keys, and the Airflow REST API password are currently stored in a `.env` file excluded from version control via `.gitignore`. This is adequate for a single-developer environment but insufficient for production, where secrets must be rotated, audited, and never stored on disk in plaintext.

A secrets management solution such as HashiCorp Vault or cloud-native equivalents (AWS Secrets Manager, Azure Key Vault) would provide encrypted storage, automatic rotation, and access logging. Docker Compose secrets or Kubernetes Secrets would inject credentials at runtime, eliminating environment files entirely.

8.4 Load Testing

The system has been validated with the synthetic dataset of 243,000 events replayed at accelerated speed, but it has not been tested under sustained production-equivalent load. A load testing phase would establish the throughput ceiling of each service, identify the first bottleneck in the pipeline, and determine the point at which consumer lag begins to accumulate.

The existing event simulator can serve as the load generator with minimal modification: adjusting the inter-event delay to simulate peak branch hours across 150 branches. The Prometheus instrumentation (Section 6.7) already captures the metrics needed to evaluate load test results — throughput, p95 latency, and consumer lag — without additional tooling.

8.5 Data Retention and Archival

The current PostgreSQL schema retains all transactions and alerts indefinitely. In a production deployment, regulatory requirements and storage constraints necessitate a data retention policy. Tunisian banking regulations require that transaction records be preserved for a minimum period, after which they may be archived to cold storage or purged.

A retention strategy would partition the `transactions` and `alerts` tables by month using PostgreSQL's native table partitioning, enabling efficient archival of older partitions to compressed cold storage while keeping recent data queryable at full speed. The batch profile rebuild (Chapter 5) would be scoped to the retention window rather than the full transaction history, reducing nightly computation time as the dataset grows.

General Conclusion

This internship project set out to design and build a real-time anomaly detection platform for branch teller operations at Amen Bank, covering four operation types: retrait, versement, virement, and remise de chèques. The result is a working end-to-end system that ingests transaction events, computes behavioral features, scores them with two complementary deep learning models, applies regulatory rules, and surfaces explainable alerts to branch supervisors through an interactive dashboard.

The platform monitors two independent actors — the client initiating the transaction and the employee processing it — reflecting the dual-actor nature of branch teller operations where anomalies can originate from either side. A streaming architecture built on RedPanda chains four microservices through message topics, supported by a hot/cold data layer (Redis and PostgreSQL) and a batch pipeline for nightly profile reconstruction. The Autoencoder detects point anomalies in individual transactions while the LSTM identifies sequential deviations in client behavior over time, with a sigmoid-weighted fusion mechanism that adapts to account maturity. Every alert carries a human-readable SHAP explanation generated by deterministic templates, enabling supervisors to understand and act on the system’s output without requiring machine learning expertise.

Beyond the detection pipeline, the project invested heavily in the infrastructure that makes a machine learning system maintainable. MLflow tracks every training experiment. An automated promotion gate ensures that only models matching or exceeding current production performance are deployed. Three independent retraining triggers — a weekly schedule, a Prometheus-driven drift detector, and a supervisor feedback divergence monitor — close the loop between model serving and model improvement. Structured JSON logging, Prometheus metrics, and Grafana dashboards provide the observability layer needed to diagnose issues without inspecting individual events.

The system was validated on a synthetic dataset of 243,000 events generated from five client archetypes calibrated against Amen Bank’s published data. The validation confirms that the pipeline works end-to-end — from event ingestion to supervisor alert — and that the retraining subsystem can detect drift and trigger model updates autonomously. The synthetic data caveat is explicit: these results demonstrate engineering readiness, not production-grade detection accuracy. The architecture is designed so that replacing synthetic data with real transactions requires no code changes — the same feature computation, the same models, and the same retraining pipeline apply without modification.

This project provided the opportunity to work across the full spectrum of a machine learning system: from data engineering and model training to streaming infrastructure, MLOps, and front-end delivery. The most valuable lesson was that building a model is a small fraction of the effort — the majority of the engineering work lies in the infrastructure that ensures the model remains correct, observable, and improvable over time.

Bibliography

- [1] R. J. Bolton and D. J. Hand, “Statistical Fraud Detection: A Review,” *Statistical Science*, vol. 17, no. 3, pp. 235–255, 2002.
- [2] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, pp. 413–422, 2008.
- [3] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, “LOF: Identifying Density-Based Local Outliers,” in *Proc. ACM SIGMOD*, pp. 93–104, 2000.
- [4] M. Sakurada and T. Yairi, “Anomaly Detection Using Autoencoders with Nonlinear Dimensionality Reduction,” in *Proc. MLSDA Workshop*, pp. 4–11, 2014.
- [5] J. An and S. Cho, “Variational Autoencoder based Anomaly Detection using Reconstruction Probability,” *SNU Data Mining Center, Special Lecture on IE*, 2015.
- [6] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [7] L. Bontemps, J. McDermott, and N.-A. Le-Khac, “Collective Anomaly Detection based on Long Short-Term Memory Recurrent Neural Networks,” in *Proc. Int. Conf. Digital Forensics and Cyber Crime (ICDF2C)*, pp. 141–152, 2016.
- [8] C. C. Aggarwal, *Outlier Analysis*, 2nd ed. Springer, 2017.
- [9] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” in *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 4765–4774, 2017.
- [10] J. Kreps, N. Narkhede, and J. Rao, “Kafka: A Distributed Messaging System for Log Processing,” in *Proc. NetDB Workshop*, 2011.

Appendix A

Enriched Feature Schema

This appendix lists the 30 enriched features computed by the Enrichment Service for each transaction event. These features constitute the input vector for both the Autoencoder and the LSTM models.

Table A.1: Complete enriched feature schema (30 features)

#	Feature	Category	Description
1	amount	Raw event	Transaction amount in DT
2	hour	Raw event	Hour of day (0–23)
3	is_retrait	Raw event	One-hot: operation is retrait
4	is_versement	Raw event	One-hot: operation is versement
5	is_virement	Raw event	One-hot: operation is virement
6	is_cheque	Raw event	One-hot: operation is remise de chèques
7	is_home_branch	Raw event	1 if transaction at client's home branch
8	z_amount	Amount contrast	$(x - \mu_{30d})/\sigma_{30d}$ for this operation type
9	amount_to_mean_ratio	Amount contrast	x/μ_{30d} for this operation type
10	is_above_threshold	Amount contrast	1 if amount $\geq 10,000$ DT
11	is_near_threshold	Amount contrast	1 if amount $\in [8,000, 9,999]$ DT
12	is_round_amount	Amount contrast	1 if amount mod 1,000 = 0
13	is_near_cheque_ceiling	Amount contrast	1 if cheque amount $\in [25,000, 29,999]$ DT
14	op_type_probability	Operation contrast	Client's 30d frequency ratio for this operation type
15	is_outside_hours	Timing contrast	1 if hour $\notin [8, 16]$

Continued on next page

#	Feature	Category	Description
16	day_distance_from_preferred	Timing contrast	day of month – client’s preferred day
17	is_new_counterparty	Counterparty contrast	1 if beneficiary/emitter not in known set
18	tx_count_24h	Velocity	Client transactions in last 24 hours
19	tx_count_7d	Velocity	Client transactions in last 7 days
20	cumulative_amount_24h	Velocity	Sum of client amounts in last 24 hours
21	has_duplicate_recent	Velocity	1 if same amount + operation in last 24h
22	near_threshold_count_7d	Velocity	Events in [8k, 10k) DT band in last 7 days
23	employee_tx_count_24h	Employee	Employee’s transaction count in last 24h
24	same_employee_client_count_24h	Employee	Times this employee served this client in 24h
25	is_salaried	Context	One-hot: archetype is salaried
26	is_small_business	Context	One-hot: archetype is small business
27	is_student	Context	One-hot: archetype is student
28	is_retiree	Context	One-hot: archetype is retiree
29	is_big_business	Context	One-hot: archetype is big business
30	account_age_days	Context	Days since account opening

Features 8–9 use a three-level fallback chain when statistics are unavailable for the current operation type: 30-day statistics, then 90-day statistics, then the archetype baseline. Feature 13 applies only to cheque operations and is zero for all other operation types. Features 18–22 are computed from the Redis rolling buffer, not from the PostgreSQL profile, ensuring they reflect the most recent events with sub-second latency.

Appendix B

Docker Compose Service Inventory

This appendix lists all 20 Docker Compose services that constitute the deployed system.

Table B.1: Docker Compose service inventory

Service	Image / Build	Port	Role
<i>Infrastructure</i>			
redpanda	vectorized/redpanda	9092	Kafka-compatible message broker
redis	redis:7	6379	Hot profile store and sequence buffers
postgres	postgres:17	5432	Cold store, alerts, evaluation labels, Airflow metadata
<i>Streaming Pipeline</i>			
enrichment-service	Custom (Dockerfile.api)	9100	Enriches raw events with 30 contrast features
scoring-service	Custom (Dockerfile.api)	9101	AE + LSTM inference, score fusion, SHAP
decision-service	Custom (Dockerfile.api)	9102	Rule evaluation, tier assignment
raw-events-sink	Custom (Dockerfile.api)	—	Writes raw events to PostgreSQL
decisions-sink	Custom (Dockerfile.api)	—	Writes alerts to PostgreSQL and risk history to Redis
<i>API and Dashboard</i>			
simulation-api	Custom (Dockerfile.api)	8000	FastAPI: scoring, feedback, health endpoints

Continued on next page

Service	Image / Build	Port	Role
dashboard	Custom (Dockerfile.api)	8501	Streamlit supervisor dashboard
<i>Orchestration</i>			
airflow-webserver	Custom (Dockerfile.airflow)	8080	Airflow UI and REST API
airflow-scheduler	Custom (Dockerfile.airflow)	—	DAG scheduling (LocalExecutor)
<i>MLOps</i>			
mlflow	ghcr.io/mlflow/mlflow	5001	Experiment tracking, artifact store
webhook-relay	Custom (Dockerfile.webhook)	—	AlertManager → Airflow REST API bridge
<i>Monitoring</i>			
prometheus	prom/prometheus	9090	Metrics collection and alert rule evaluation
alertmanager	prom/alertmanager	9093	Alert deduplication and webhook routing
grafana	grafana/grafana	3000	Metrics visualization dashboards
kafka-exporter	danielqsj/kafka-exporter	9308	Broker-level metrics (consumer lag)
<i>On-Demand (profile-gated)</i>			
hydrate-redis	Custom (Dockerfile.api)	—	Seeds Redis profiles from PostgreSQL at startup
simulator	Custom (Dockerfile.api)	—	Replays synthetic events into RedPanda

Services are organized into six functional groups. The three streaming pipeline services and two sink consumers form the real-time processing chain. Infrastructure services (RedPanda, Redis, PostgreSQL) are shared across all components. The on-demand services (`hydrate-redis` and `simulator`) run once at startup or on explicit invocation using Docker Compose profiles, and exit after completion.

All custom services share a common `Dockerfile.api` base image except the Airflow containers (`Dockerfile.airflow`, which excludes PyTorch to save 2 GB), the webhook relay (`Dockerfile.webhook`, a minimal `python:3.11-slim` image at approximately 150 MB),

and the training job (launched by the DockerOperator using a separate `amen-anomaly-training:latest` image not listed here, as it runs transiently during retraining).